BUS INTERFACE

INVENTRA™

THE INTELLIGENT APPROACH TO INTELLECTUAL PROPERTY

MUSBFSFC

USB 1.1 FULL-SPEED FUNCTION CONTROLLER

Programmer's Guide

CONFIDENTIAL

SOFT CORES

03/02 PG-40104.003-FC

© 2000-2002 Mentor Graphics Corporation
All Rights Reserved

Confidential. May be photocopied by licensed customers of Mentor Graphics for internal business purposes only.

The product(s) described in this document are trade secret and proprietary products of Mentor Graphics Corporation or its licensors and are subject to license terms. No part of this document may be photocopied, reproduced or translated, disclosed or otherwise provided to third parties, without the prior written consent of Mentor Graphics.

The document is for informational and instructional purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in the written contracts between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OR MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND Use, duplication, or disclosure by the Government is subject to restrictions as set forth in the subdivision (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013.

A complete list of trademark names appears in a separate “Trademark Information” document.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070.

This is an unpublished work of Mentor Graphics Corporation.

For Customer Support on this product:

- Call up the Inventra Customer Inquiry Service at **http://www.mentor.com/supportnet**
- Email **support_net@mentor.com**
- Phone **1-877-763-8470** (toll-free in US, Mexico and Canada)
  (Customers in other parts of the world can access this service via the AT&T USA Direct service in their country.)

Full details are given in the Customer Support Handbook, provided in Adobe Acrobat format as **custhb.pdf** in the **/databook** directory on the Inventra™ Soft Cores CD. Please note the checklists of actions to take and information to have to hand when contacting Inventra Customer Support that are given in the Customer Support Handbook.

TABLE OF CONTENTS

1. INTRODUCTION 6

2. REGISTER DESCRIPTION 7
   - 2.1. MUSBFSFC Register Map 7
   - 2.2. FAddr 8
   - 2.3. Power 9
   - 2.4. IntrIn1 10
   - 2.5. IntrIn2 10
   - 2.6. IntrOut1 11
   - 2.7. IntrOut2 11
   - 2.8. IntrUSB 12
   - 2.9. IntrIn1E and IntrIn2E 12
   - 2.10. IntrOut1E and IntrOut2E 13
   - 2.11. IntrUSBE 13
   - 2.12. Frame1 14
   - 2.13. Frame2 14
   - 2.14. Index 14
   - 2.15. CSR0 14
   - 2.16. Count0 15
   - 2.17. InMaxP 15
   - 2.18. InCSR1 16
   - 2.19. InCSR2 17
   - 2.20. OutMaxP 17
   - 2.21. OutCSR1 18
   - 2.22. OutCSR2 18
   - 2.23. OutCount1 19
   - 2.24. OutCount2 19
   - 2.25. FIFOx (Addresses 20h – 2Fh) 19

3. PROGRAMMING SCHEME 20
   - 3.1. USB Interrupt Handling 20

4. USB RESET....21

5. SUSPEND/RESUME....21

5.1. MUSBFSFC Active During Suspend....21

5.2. MUSBFSFC Inactive During Suspend....21

5.3. Remote Wakeup....21

6. ENDPOINT 0 HANDLING....22

6.1. Zero Data Requests....22

6.2. Write Requests....23

6.3. Read Requests....23

6.4. Endpoint 0 States....24

6.5. Endpoint 0 Service Routine....26

6.5.1. IDLE Mode....28

6.5.2. TX Mode....29

6.5.3. RX Mode....30

6.6. Error Handling....31

7. BULK IN ENDPOINT....32

7.1. Setup....32

7.2. Operation....33

7.3. Error Handling....33

8. BULK OUT ENDPOINT....33

8.1. Setup....34

8.2. Operation....34

8.3. Error Handling....34

9. INTERRUPT IN ENDPOINT....35

10. INTERRUPT OUT ENDPOINT....35

11. ISOCHRONOUS IN ENDPOINT....35

11.1. Setup....36

11.2. Operation....36

11.3. Error Handling....37

12. ISOCHRONOUS OUT ENDPOINT 37
12.1. Setup 37
12.2. Operation 38
12.3. Error Handling 38

13. TRANSACTION FLOWS 39
13.1. Control Transactions 39
13.1.1. Setup Phase 39
13.1.2. IN Data Phase 40
... and following Status Phase 41
13.1.3. Out Data Phase 42
... and following Status Phase 43
13.2. Bulk/Interrupt Transactions 44
13.2.1. IN Transaction 44
13.2.2. OUT Transaction 45
13.3. Isochronous Transactions 46
13.3.1. IN Transaction 46
13.3.2. OUT Transaction 47

14. FIRMWARE 48
14.1. Routines 49
14.2. Header Files 64

15. REVISION HISTORY 73
15.1. Issue 1 73
15.2. Issue 2 73

1. INTRODUCTION

The Inventra™ MUSBFSFC is a user-configurable core that complies with the USB 2.0 (and earlier) specifications for Full-speed (12 Mbps) functions.

The core is configurable for up to 15 IN endpoints and for up to 15 OUT endpoints in addition to Endpoint 0. These additional endpoints can be individually programmed for Bulk/Interrupt or Isochronous transfers.

Each endpoint requires a FIFO to be associated with it. The MUSBFSFC has a RAM interface for connecting to a single block of synchronous single-port RAM which is used for all the endpoint FIFOs. (The RAM block itself needs to be added by the user.)

The MUSBFSFC's FIFO interface is configurable for an Endpoint 0 FIFO that is 8, 16, 32 or 64 bytes deep and buffers 1 packet. The FIFO interface is also configurable with regard to the other endpoint FIFOs which may be from 8 to 2048 bytes in size and can buffer either 1 or 2 packets. Separate FIFOs may be associated with each endpoint: alternatively an IN endpoint and the OUT endpoint with the same Endpoint number can be configured to use the same FIFO, for example to reduce the size of RAM block needed.

The MUSBFSFC is offered with a choice of high-level CPU interfaces. In the base model, access to the FIFOs and the internal control/status registers is via an 8-bit PVCI[1]-compatible synchronous CPU interface. However, the MUSBFSFC can readily be connected to a range of standard buses through bridges such as the MUSBFSFC – AHB bridge provided with the core (and described in the separate MUSBFSFC – AHB Bridge specification provided in the **musbfsfc/docs** directory as **musbfsfc_ahb_an.pdf**).

There is also support for DMA access to the Endpoint FIFOs. The optional bridges may also offer DMA support in which case this is described in the appropriate Bridge Specification.

The MUSBFSFC provides all the USB packet encoding, decoding and checking – interrupting the CPU only when endpoint data has been successfully transferred.

A graphical user interface is provided for configuring the core to the user's requirements. The script to use depends on the CPU interface that is selected.

This document describes the software interface to the MUSBFSFC and gives examples of how a typical application should drive the core to produce a USB-compliant function.

[1] Peripheral Virtual Component Interface, as defined by the VSI Alliance™ Virtual Component Interface Standard (OCB 2 1.0).

© 2000-2002 Mentor Graphics Corporation.

2. REGISTER DESCRIPTION

2.1. MUSBFSFC REGISTER MAP

The MUSBFSFC register map is divided into three sections:

1. Common USB registers (addresses 00h to 0Fh). These registers provide control and status for the entire function controller.

2. Indexed registers (addresses 10h to 1Fh). These registers provide control and status information for the different endpoints. There is an InMaxP and two InCSR registers for each IN endpoint and an OutMaxP, two OutCSR, and two OutCount registers for each OUT endpoint (except for Endpoint 0 which has a reduced registered set: see below). Only the registers for one IN endpoint and the corresponding OUT endpoint (i.e. the one with the same endpoint number) appear in the register map at any one time. The endpoints are selected by writing the endpoint number to the Index register. So to access the registers for IN Endpoint 1 and OUT Endpoint 1, 1 must first be written to the Index register: the appropriate control and status registers then appear in the memory map.

3. FIFOs (addresses 20h to 2F). The FIFOs for the IN endpoints appear as single bytes, consecutively in the memory map starting at address 20h. The FIFOs for the OUT endpoints also appear consecutively at the same set of addresses. A write to address 21h results in a byte being loaded into the FIFO for IN Endpoint 1. A read of address 21h results in a byte being unloaded from the FIFO for OUT Endpoint 1.

The MUSBFSFC register map is shown below. **Note:** Any additional registers associated with any bridge provided for use with the MUSBFSFC core or any changes to the following registers that result from using this bridge are described in the separate specification for the bridge included in the **musbfsfc/docs** directory.

MUSBFSFC REGISTER MAP – Common USB registers		
ADDR	**NAME**	**DESCRIPTION**
00	FAddr	Function address register.
01	Power	Power management register.
02	IntrIn1	Interrupt register for Endpoint 0 plus IN Endpoints 1 to 7.
03	IntrIn2	Interrupt register for IN Endpoints 8 to 15. Only present if more than 7 IN endpoints are included.
04	IntrOut1	Interrupt register for OUT Endpoints 1 to 7.
05	IntrOut2	Interrupt register for OUT Endpoints 8 to 15. Only present if more than 7 OUT endpoints are included.
06	IntrUSB	Interrupt register for common USB interrupts.
07	IntrIn1E	Interrupt enable register for IntrIn1.
08	IntrIn2E	Interrupt enable register for IntrIn2. Only present if more than 7 IN endpoints are included.
09	IntrOut1E	Interrupt enable register for IntrOut1.
0A	IntrOut2E	Interrupt enable register for IntrOut2. Only present if more than 7 OUT endpoints are included.
0B	IntrUSBE	Interrupt enable register for IntrUSB.
0C	Frame1	Frame number bits 0 to 7.
0D	Frame2	Frame number bits 8 to 10.
0E	Index	Index register for selecting the endpoint status and control registers.
0F	–	*Unused*, always returns 0.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

MUSBFSFC REGISTER MAP – Indexed registers (Control Status registers for endpoint selected by the Index register)		
10	InMaxP	Maximum packet size for IN endpoint. (Index register set to select Endpoints 1 – 15 only)
11	CSR0	Control Status register for Endpoint 0. (Index register set to select Endpoint 0)
	InCSR1	Control Status register 1 for IN endpoint. (Index register set to select Endpoints 1 – 15)
12	InCSR2	Control Status register 2 for IN endpoint. (Index register set to select Endpoints 1 – 15 only)
13	OutMaxP	Maximum packet size for OUT endpoint. (Index register set to select Endpoints 1 – 15 only)
14	OutCSR1	Control Status register 1 for OUT endpoint. (Index register set to select Endpoints 1 – 15 only)
15	OutCSR2	Control Status register 2 for OUT endpoint. (Index register set to select Endpoints 1 – 15 only)
16	Count0	Number of received bytes in Endpoint 0 FIFO. (Index register set to select Endpoint 0)
	OutCount1	Number of bytes in OUT endpoint FIFO (lower byte). (Index register set to select Endpoints 1 – 15)
17	OutCount2	Number of bytes in OUT endpoint FIFO (upper byte). (Index register set to select Endpoints 1 – 15 only)
18–1F	–	*Unused*, always return 0.
MUSBFSFC REGISTER MAP – FIFOs		
20–2Fh	FIFOx	FIFOs for Endpoints 0 to 15.

Note: In the following bit descriptions:

'r' means that the bit is read only
'set' means that the bit can only be written to set it
'clear' means that the bit can only be written to clear it
'rw' means that the bit can be both read and written
'r/set' means that the bit can be read or set but it can't be cleared
'r/clear' means that the bit can be read or cleared but it can't be set
'self-clearing' means the bit will be cleared automatically when the associated action has been executed.

2.2. FADDR

FAddr is an 8-bit register that should be written with the function's 7-bit address. It is then used for decoding the function address in subsequent token packets.

Address: 00h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	Update	**Function Address**						
From CPU	r	rw	rw	rw	rw	rw	rw	rw
From USB	r/clear	r	r	r	r	r	r	r

Bit	Name	Function
D7	**Update**	Set when FAddr is written. Cleared when the new address takes effect (at the end of the current transfer).
D6–D0	**Func Addr**	The function address.

© 2000-2002 Mentor Graphics Corporation.

This register should be written with the address value contained in the SET_ADDRESS standard device request (see *Universal Serial Bus Specification* Revision 2.0, Chapter 9), when it is received on Endpoint 0. The new address will not take effect immediately as the host will still be using the old address for the Status stage of the device request. The MUSBFSFC will continue to use the old address for decoding packets until the device request has completed. The status of the device request can be determined by reading bit 7 of this register. When a new address is written to this register, bit 7 will be automatically set. It will remain high until the device request has completed and will be cleared when the new address takes effect.

2.3. POWER

Power is an 8-bit register that is used for controlling Suspend and Resume signaling.

Address: 01h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	ISO Update	–	–	–	**Reset**	**Resume**	**Suspend Mode**	**Enable Suspend**
From CPU	rw	r	r	r	r	rw	r	rw
From USB	r	r	r	r	rw	r	set	r

Bit	Name	Function
D7	**ISO Update**	When set by the CPU the MUSBFSFC will wait for an SOF token from the time InPktRdy is set before sending the packet. If an IN token is received before an SOF token, then a zero length data packet will be sent. *This bit is only used by endpoints performing Isochronous transfers.*
D6 – D4	**–**	*Unused*, always return zero.
D3	**Reset**	This read only bit is set while Reset signaling is present on the bus.
D2	**Resume**	Set by the CPU to generate Resume signaling when the function is in Suspend mode. The CPU should clear this bit after 10 ms (a maximum of 15 ms) to end Resume signaling.
D1	**Suspend Mode**	Set by the USB when Suspend mode is entered. Cleared when the CPU reads the interrupt register, or sets the Resume bit of this register.
D0	**Enable Suspend**	Set by the CPU to enable entry into Suspend mode when Suspend signaling is received on the bus.

The **ISO Update** bit affects all IN Isochronous endpoints in the MUSBFSFC. It is normally used as a method of ensuring a "clean" start-up of an IN Isochronous pipe. See the section on IN Isochronous Endpoints (Section 11) for more details on starting up IN Isochronous pipes.

The **Reset** bit can be used to determine when Reset signaling is present on the USB. It will go high when Reset signaling is detected and remain high until the bus reverts to an idle state.

The **Resume** bit is used to force the MUSBFSFC to generate Resume signaling on the USB to perform remote wake-up from Suspend mode. Once set high, it should be left high for approximately 10 ms (at least 1 ms and no more than 15 ms), then cleared.

The **Suspend Mode** bit is set by the MUSBFSFC when Suspend mode is entered. It will be cleared when the IntrUSB register is read (as a result of receiving a Suspend interrupt). It will also be cleared if Suspend mode is left by setting the Resume bit to initiate a remote wake-up.

The **Enable Suspend** bit is set to enable entry into Suspend mode when Suspend signaling is received on the bus.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

2.4. INTRIN1

IntrIn1 is an 8-bit read-only register that indicates which of the interrupts for IN Endpoints 1 – 7 are currently active. It also indicates whether the Endpoint 0 interrupt is currently active. *Note:* All active interrupts will be cleared when this register is read.

Address: 02h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	EP7	**EP6**	**EP5**	**EP4**	**EP3**	**EP2**	**EP1**	**EP0**
From CPU	r	r	r	r	r	r	r	r
From USB	set	set	set	set	set	set	set	set

Bit	Name	Function
D7	**EP7**	IN Endpoint 7 interrupt.
D6	**EP6**	IN Endpoint 6 interrupt.
D5	**EP5**	IN Endpoint 5 interrupt.
D4	**EP4**	IN Endpoint 4 interrupt.
D3	**EP3**	IN Endpoint 3 interrupt.
D2	**EP2**	IN Endpoint 2 interrupt.
D1	**EP1**	IN Endpoint 1 interrupt.
D0	**EP0**	Endpoint 0 interrupt.

2.5. INTRIN2

IntrIn2 is an 8-bit read-only register that indicates which of the interrupts for IN Endpoints 8 – 15 are currently active. This register will only be present if the MUSBFSFC is configured for more than 7 IN endpoints. *Note:* All active interrupts will be cleared when this register is read.

Address: 03h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	EP15	**EP14**	**EP13**	**EP12**	**EP11**	**EP10**	**EP9**	**EP8**
From CPU	r	r	r	r	r	r	r	r
From USB	set	set	set	set	set	set	set	set

Bit	Name	Function
D7	**EP15**	IN Endpoint 15 interrupt.
D6	**EP14**	IN Endpoint 14 interrupt.
D5	**EP13**	IN Endpoint 13 interrupt.
D4	**EP12**	IN Endpoint 12 interrupt.
D3	**EP11**	IN Endpoint 11 interrupt.

© 2000-2002 Mentor Graphics Corporation.

Bit	Name	Function
D2	**EP10**	IN Endpoint 10 interrupt.
D1	**EP9**	IN Endpoint 9 interrupt.
D0	**EP8**	IN Endpoint 8 interrupt.

2.6. INTROUT1

IntrOut1 is an 8-bit read-only register that indicates which of the interrupts for OUT Endpoints 1 – 7 are currently active. *Note:* All active interrupts will be cleared when this register is read.

Address: 04h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	EP7	**EP6**	**EP5**	**EP4**	**EP3**	**EP2**	**EP1**	–
From CPU	r	r	r	r	r	r	r	r
From USB	set	set	set	set	set	set	set	–

Bit	Name	Function
D7	**EP7**	OUT Endpoint 7 interrupt.
D6	**EP6**	OUT Endpoint 6 interrupt.
D5	**EP5**	OUT Endpoint 5 interrupt.
D4	**EP4**	OUT Endpoint 4 interrupt.
D3	**EP3**	OUT Endpoint 3 interrupt.
D2	**EP2**	OUT Endpoint 2 interrupt.
D1	**EP1**	OUT Endpoint 1 interrupt.
D0	–	Unused, always returns 0

2.7. INTROUT2

IntrOut2 is an 8-bit read-only register that indicates which of the interrupts for OUT Endpoints 8 – 15 are currently active. This register will only be present if the MUSBFSFC is configured for more than 7 OUT endpoints. *Note:* All active interrupts are cleared when this register is read.

Address: 05h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	EP15	**EP14**	**EP13**	**EP12**	**EP11**	**EP10**	**EP9**	**EP8**
From CPU	r	r	r	r	r	r	r	r
From USB	set	set	set	set	set	set	set	set

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

Bit	Name	Function
D7	**EP15**	OUT Endpoint 15 interrupt.
D6	**EP14**	OUT Endpoint 14 interrupt.
D5	**EP13**	OUT Endpoint 13 interrupt.
D4	**EP12**	OUT Endpoint 12 interrupt.
D3	**EP11**	OUT Endpoint 11 interrupt.
D2	**EP10**	OUT Endpoint 10 interrupt.
D1	**EP9**	OUT Endpoint 9 interrupt.
D0	**EP8**	OUT Endpoint 8 interrupt.

2.8. INTRUSB

IntrUSB is an 8-bit read-only register that indicates which USB interrupts are currently active. All active interrupts will be cleared when this register is read.

Address: 06h; *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	–	–	–	–	**SOF**	**Reset**	**Resume**	**Suspend**
From CPU	r	r	r	r	r	r	r	r
From USB	–	–	–	–	set	set	set	set

Bit	Name	Function
D7 – D4	**–**	*Unused*, always return 0.
D3	**SOF**	Set at the start of each frame.
D2	**Reset**	Set when Reset signaling is detected on the bus.
D1	**Resume**	Set when Resume signaling is detected on the bus while the MUSBFSFC is in Suspend mode..
D0	**Suspend**	Set when Suspend signaling is detected on the bus.

2.9. INTRIN1E AND INTRIN2E

IntrIn1E and IntrIn2E are 8-bit registers that provide interrupt enable bits for the interrupts in IntrIn1 and IntrIn2 respectively. On reset, the bits corresponding to Endpoint 0 and the IN endpoints included in the design are set to 1, while the remaining bits are set to 0.

IntrIn1E *Address:* 07h; *Reset value:*8'hFF masked with the IN endpoints implemented (including Endpoint 0)

	D7	D6	D5	D4	D3	D2	D1	D0
	EP7	**EP6**	**EP5**	**EP4**	**EP3**	**EP2**	**EP1**	**EP0**
From CPU	rw	rw	rw	rw	rw	rw	rw	rw
From USB	r	r	r	r	r	r	r	r

© 2000-2002 Mentor Graphics Corporation.

IntrIn2E *Address:* 08h; *Reset value:*8'hFF masked with the IN endpoints implemented

	D7	D6	D5	D4	D3	D2	D1	D0
	EP15	**EP14**	**EP13**	**EP12**	**EP11**	**EP10**	**EP9**	**EP8**
From CPU	rw	rw	rw	rw	rw	rw	rw	rw
From USB	r	r	r	r	r	r	r	r

2.10. INTROUT1E AND INTROUT2E

IntrOut1 and IntrOut2 are 8-bit registers that provide interrupt enable bits for the interrupts in IntrOut1 and IntrOut2 respectively. On reset, the bits corresponding to the OUT endpoints included in the design are set to 1, while the remaining bits are set to 0.

IntrOut1E *Address:* 09h; *Reset value:* 8'hFE masked with the OUT endpoints implemented

	D7	D6	D5	D4	D3	D2	D1	D0
	EP7	**EP6**	**EP5**	**EP4**	**EP3**	**EP2**	**EP1**	**–**
From CPU	rw	rw	rw	rw	rw	rw	rw	r
From USB	r	r	r	r	r	r	r	r

IntrOut2E *Address:* 0Ah; *Reset value:* 8'hFF masked with the OUT endpoints implemented

	D7	D6	D5	D4	D3	D2	D1	D0
	EP15	**EP14**	**EP13**	**EP12**	**EP11**	**EP10**	**EP9**	**EP8**
From CPU	rw	rw	rw	rw	rw	rw	rw	rw
From USB	r	r	r	r	r	r	r	r

2.11. INTRUSBE

IntrUSBE is an 8-bit register that provides interrupt enable bits for each of the interrupts in IntrUSB.

Address: 0Bh; *Reset value:* 8'h06

	D7	D6	D5	D4	D3	D2	D1	D0
	–	**–**	**–**	**–**	**SOF**	**Reset**	**Resume**	**Suspend**
From CPU	r	r	r	r	rw	rw	rw	rw
From USB	r	r	r	r	r	r	r	r

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

2.12. FRAME1

Frame1 is an 8-bit read-only register that holds the lower 8 bits of the last received frame number.

*Address:*0Ch; *Reset value:* 8'b00000000

	D7	...	D0
	Lower 8 bits of Frame Number		
From CPU	r	...	r
From USB	rw	...	rw

2.13. FRAME2

Frame2 is a 3-bit read-only register that holds the upper 3 bits of the last received frame number.

*Address:*0Dh; *Reset value:*3'b000

	D2	...	D0
	Upper 3 bits of Frame Number		
From CPU	r	...	r
From USB	rw	...	rw

2.14. INDEX

Index is a 4-bit register that determines which endpoint control/status registers are accessed at addresses 10h to 17h.

Address: 0Eh; *Reset value:*4'b0000

	D3	D2	D1	D0
	Selected Endpoint			
From CPU	rw	rw	rw	rw
From USB	r	r	r	r

Each IN endpoint and each OUT endpoint have their own set of control/status registers. Only one set of IN control/status and one set of OUT control/status registers appear in the memory map at any one time. Before accessing an endpoint's control/status registers, the endpoint number should be written to the Index register to ensure that the correct control/status registers appear in the memory map.

2.15. CSR0

CSR0 is an 8-bit register that provides control and status bits for Endpoint 0.

Address: 11h (with the Index register set to 0); *Reset value:* 8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	Serviced SetupEnd *(self-clearing)*	**Serviced OutPktRdy** *(self-clearing)*	**SendStall** *(self-clearing)*	**SetupEnd**	**DataEnd** *(self-clearing)*	**SentStall**	**InPktRdy** *(self-clearing)*	**OutPktRdy**
From CPU	set	set	set	r	set	r/clear	r/set	r
From USB	r	r	r	set	r	set	r	set

SOFT CORES

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

Bit	Name	Function
D7	**ServicedSetupEnd**	The CPU writes a 1 to this bit to clear the SetupEnd bit. It is cleared automatically.
D6	**ServicedOutPktRdy**	The CPU writes a 1 to this bit to clear the OutPktRdy bit. It is cleared automatically.
D5	**SendStall**	The CPU writes a 1 to this bit to terminate the current transaction. The STALL handshake will be transmitted and then this bit will be cleared automatically.
D4	**SetupEnd**	This bit will be set when a control transaction ends before the DataEnd bit has been set. An interrupt will be generated and the FIFO flushed at this time. The bit is cleared by the CPU writing a 1 to the ServicedSetupEnd bit.
D3	**DataEnd**	The CPU sets this bit: 1. When setting InPktRdy for the last data packet. 2. When clearing OutPktRdy after unloading the last data packet. 3. When setting InPktRdy for a zero length data packet. It is cleared automatically.
D2	**SentStall**	This bit is set when a STALL handshake is transmitted. The CPU should clear this bit.
D1	**InPktRdy**	The CPU sets this bit after loading a data packet into the FIFO. It is cleared automatically when the data packet has been transmitted. An interrupt is generated when the bit is cleared.
D0	**OutPktRdy**	This bit is set when a data packet has been received. An interrupt is generated when this bit is set. The CPU clears this bit by setting the ServicedOutPktRdy bit.

CSR0 is used for all control/status of Endpoint 0. For details of how to service device requests to Endpoint 0, see Section 5: 'Endpoint 0 Handling'.

2.16. COUNT0

Count0 is a 7-bit read-only register that indicates the number of received data bytes in the Endpoint 0 FIFO. The value returned is valid while OutPktRdy (CSR0.D0) is set.

Address: 16h (with the Index register set to 0); *Reset value:*7'h00

	D6	D5	D4	D3	D2	D1	D0
	Endpoint 0 OUT Count						
From CPU	r	r	r	r	r	r	r
From USB	w	w	w	w	w	w	w

2.17. INMAXP

InMaxP is an 8-bit register that holds the maximum packet size for transactions through the currently-selected IN endpoint – in units of 8 bytes, except that a value of 128 sets the maximum packet size to 1023 (the maximum size for an Isochronous packet transferred in a Full-speed transaction) rather than 1024. In setting this value, you should note the constraints placed by the USB Specification on packet sizes for Bulk, Interrupt and Isochronous transactions in Full-speed operations.

There is an InMaxP register for each IN endpoint (except Endpoint 0).

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

Address: 10h; *Reset value:* 8'h00

	D7	...	D0
	Maximum Packet Size/transaction		
From CPU	rw	...	rw
From USB	r	...	r

The value written to this register should match the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the associated endpoint (see *Universal Serial Bus Specification* Revision 2.0, Chapter 9). A mismatch could cause unexpected results.

If a value greater than the configured IN FIFO size for the endpoint is written to this register, the value will be automatically changed to the IN FIFO size. If the value written to this register is less than, or equal to, half the IN FIFO size, two IN packets can be buffered.

The register is reset to 0. If this register is changed after packets have been sent from the endpoint, the endpoint IN FIFO should be completely flushed (using the FlushFIFO bit (D3) in InCSR1) after writing the new value to this register.

2.18. INCSR1

InCSR1 is an 8-bit register that provides control and status bits for IN transactions through the currently-selected endpoint. There is an InCSR1 register for each IN endpoint (not including Endpoint 0).

Address: 11h; *Reset value:*8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	–	**ClrDataTog**	**SentStall**	**SendStall**	**FlushFIFO** *(self-clearing)*	**UnderRun**	**FIFO NotEmpty**	**InPktRdy**
From CPU	r	set	r/clear	rw	set	r/clear	r/clear	r/set
From USB	r	r/clear	set	r	r	set	set	clear

Bit	Name	Function
D7	**–**	*Unused.* Returns zero when read.
D6	**ClrDataTog**	The CPU writes a 1 to this bit to reset the endpoint IN data toggle to 0.
D5	**SentStall**	This bit is set when a STALL handshake is transmitted. The FIFO is flushed and the InPktRdy bit is cleared (see below). The CPU should clear this bit.
D4	**SendStall**	The CPU writes a 1 to this bit to issue a STALL handshake to an IN token. The CPU clears this bit to terminate the stall condition. *This bit has no effect if the IN endpoint is in ISO mode.*
D3	**FlushFIFO**	The CPU writes a 1 to this bit to flush the next packet to be transmitted from the endpoint IN FIFO. The FIFO pointer is reset and the InPktRdy bit (below) is cleared. *Note:* If the FIFO contains two packets, FlushFIFO will need to be set twice to completely clear the FIFO.
D2	**UnderRun**	In ISO mode, this bit is set when a zero length data packet is sent after receiving an IN token with the InPktRdy bit not set. In Bulk/Interrupt mode, this bit is set when a NAK is returned in response to an IN token. The CPU should clear this bit.
D1	**FIFONotEmpty**	This bit is set when there is at least 1 packet in the IN FIFO.
D0	**InPktRdy**	The CPU sets this bit after loading a data packet into the FIFO. It is cleared automatically when a data packet has been transmitted. An interrupt is generated (if enabled) when the bit is cleared.

© 2000-2002 Mentor Graphics Corporation.

2.19. INCSR2

InCSR2 is an 8-bit register that provides further control bits for IN transactions through the currently-selected endpoint. There is an InCSR2 register for each IN endpoint (not including Endpoint 0).

Address: 12h; *Reset value:*8'h20

	D7	D6	D5	D4	D3	D2	D1	D0
	AutoSet	**ISO**	**Mode**	**DMAEnab**	**FrcDataTog**	–	–	–
From CPU	rw	rw	rw	rw	rw	r	r	r
From USB	r	r	r	r	r	r	r	r

Bit	Name	Function
D7	**AutoSet**	If the CPU sets this bit, InPktRdy will be automatically set when data of the maximum packet size (value in InMaxP) is loaded into the IN FIFO. If a packet of less than the maximum packet size is loaded, then InPktRdy will have to be set manually. When 2 packets are in the IN FIFO then InPktRdy will also be automatically set when the first packet has been sent, if the second packet is the maximum packet size.
D6	**ISO**	The CPU sets this bit to enable the IN endpoint for isochronous transfers (ISO mode), and clears it to enable the IN endpoint for bulk/interrupt transfers.
D5	**Mode**	The CPU sets this bit to enable the endpoint direction as IN, and clears it to enable the endpoint direction as OUT. *Valid only where the same endpoint FIFO is used for both IN and OUT transactions.*
D4	**DMAEnab**	The CPU sets this bit to enable the DMA request for the IN endpoint.
D3	**FrcDataTog**	The CPU sets this bit to force the endpoint's IN data toggle to switch after each data packet is sent regardless of whether an ACK was received. This can be used by interrupt IN endpoints which are used to communicate rate feedback for Isochronous endpoints.
D2–D0	**–**	Unused, always return 0.

2.20. OUTMAXP

OutMaxP is an 8-bit register that holds the maximum packet size for transactions through the currently-selected OUT endpoint –in units of 8 bytes, except that a value of 128 sets the maximum packet size to 1023 (the maximum size for an isochronous packet) rather than 1024. In setting this value, you should note the constraints placed by the USB Specification on packet sizes for Bulk, Interrupt and Isochronous transactions in Full-speed operations.

There is an OutMaxP register for each OUT endpoint (except Endpoint 0).

Address: 13h *Reset value:* 8'h00

	D7	...	D0
	Maximum Packet Size/transaction		
From CPU	rw	...	rw
From USB	r	...	r

The value written to this register should match the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the associated endpoint (see *Universal Serial Bus Specification* Revision 2.0, Chapter 9). A mismatch could cause unexpected results.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

The total amount of data represented by the value written to this register must not exceed the FIFO size for the OUT endpoint, and should not exceed half the FIFO size if double-buffering is required. If a value greater than the configured OUT FIFO size for the endpoint is written to this register, the value will be automatically changed to the OUT FIFO size. If the value written to this register is less than, or equal to, half the OUT FIFO size, two OUT packets can be buffered.

2.21. OUTCSR1

OutCSR1 is an 8-bit register that provides control and status bits for OUT transactions through the currently-selected endpoint. It is reset to 0.

*Address:*14h; *Reset value:*8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	ClrDataTog	**SentStall**	**SendStall**	**FlushFIFO** *(self-clearing)*	**DataError**	**OverRun**	**FIFOFull** *(self-clearing)*	**OutPktRdy**
From CPU	set	r/clear	rw	set	r	r/clear	r	r/clear
From USB	r/clear	set	r	r	set	set	set	set

Bit	Name	Function
D7	**ClrDataTog**	The CPU writes a 1 to this bit to reset the endpoint data toggle to 0.
D6	**SentStall**	This bit is set when a STALL handshake is transmitted. The CPU should clear this bit.
D5	**SendStall**	The CPU writes a 1 to this bit to issue a STALL handshake. The CPU clears this bit to terminate the stall condition. *This bit has no effect if the OUT endpoint is in ISO mode.*
D4	**FlushFIFO**	The CPU writes a 1 to this bit to flush the next packet to be read from the endpoint OUT FIFO. *Note:* If the FIFO contains two packets, FlushFIFO will need to be set twice to completely clear the FIFO.
D3	**DataError**	This bit is set when OutPktRdy is set if the data packet has a CRC or bit-stuff error. it is cleared when OutPktRdy is cleared. *The bit is only valid in ISO mode.*
D2	**OverRun**	This bit is set if an OUT packet cannot be loaded into the OUT FIFO. The CPU should clear this bit. *The bit is only valid in ISO mode.*
D1	**FIFOFull**	This bit is set when no more packets can be loaded into the OUT FIFO.
D0	**OutPktRdy**	This bit is set when a data packet has been received. The CPU should clear this bit when the packet has been unloaded from the OUT FIFO. An interrupt is generated when the bit is set.

2.22. OUTCSR2

OutCSR2 is an 8-bit register that provides further control bits for OUT transactions through the currently-selected endpoint. It is reset to 0.

*Address:*15h; *Reset value:*8'h00

	D7	D6	D5	D4	D3	D2	D1	D0
	AutoClear	**ISO**	**DMAEnab**	**DMAMode**	–	–	–	–
From CPU	rw	rw	rw	rw	r	r	r	r
From USB	r	r	r	r	r	r	r	r

© 2000-2002 Mentor Graphics Corporation.

Bit	Name	Function
D7	**AutoClear**	If the CPU sets this bit then the OutPktRdy bit will be automatically cleared when a packet of OutMaxP bytes has been unloaded from the OUT FIFO. When packets of less than the maximum packet size are unloaded, OutPktRdy will have to be cleared manually.
D6	**ISO**	The CPU sets this bit to enable the OUT endpoint for Isochronous transfers, and clears it to enable the OUT endpoint for Bulk/Interrupt transfers.
D5	**DMAEnab**	The CPU sets this bit to enable the DMA request for the OUT endpoint.
D4	**DMAMode**	Two modes of DMA operation are supported: DMA Mode 0 in which a DMA request is generated for all received packets, together with an interrupt (if enabled); and DMA Mode 1 in which a DMA request (but no interrupt) is generated for OUT packets of size OutMaxP bytes and an interrupt (but no DMA request) is generated for OUT packets of any other size. The CPU sets this bit to select DMA Mode 1 and clears this bit to select DMA Mode 0.
D3 – D0	–	*Unused*, always return 0.

2.23. OUTCOUNT1

OutCount1 is a 8-bit read-only register that holds the lower 8 bits of the number of received data bytes in the packet in the FIFO associated with the currently-selected OUT endpoint. The value returned is valid while OutPktRdy (OutCSR1.D0) is set.

Address: 16h; *Reset value:* 8'b00000000

	D7	...	D0
	Endpoint OUT Count – lower 8 bits		
From CPU	r	...	r
From USB	w	...	w

2.24. OUTCOUNT2

OutCount2 is a 3-bit read-only register that holds the upper 3 bits of the number of received data bytes in the packet in the FIFO associated with the currently-selected OUT endpoint. The value returned is valid while OutPktRdy (OutCSR1.D0) is set.

Address: 17h; *Reset value:* 3'b000

	D2	...	D0
	Endpoint OUT Count – upper 3 bits		
From CPU	r	...	r
From USB	w	...	w

2.25. FIFOx (Addresses 20h – 2Fh)

This address range provides 16 addresses for CPU access to the FIFOs for each endpoint. Writing to these addresses loads data into the IN FIFO for the corresponding endpoint. Reading from these addresses unloads data from the OUT FIFO for the corresponding endpoint.

The FIFOs are located on byte boundaries (Endpoint 0 at 20h, Endpoint 1 at 21h ... Endpoint 15 at 2Fh).

Note: Using the MUSBFSFC – AHB bridge requires a different address mapping to be used for these FIFOs. Details are given in the separate specification for that bridge included in the **musbfsfc/docs** directory.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

3. PROGRAMMING SCHEME

This and the following sections look at the actions that the device controlling the MUSBFSFC core will need to perform and at the aspects of the operation of the core that affect this.

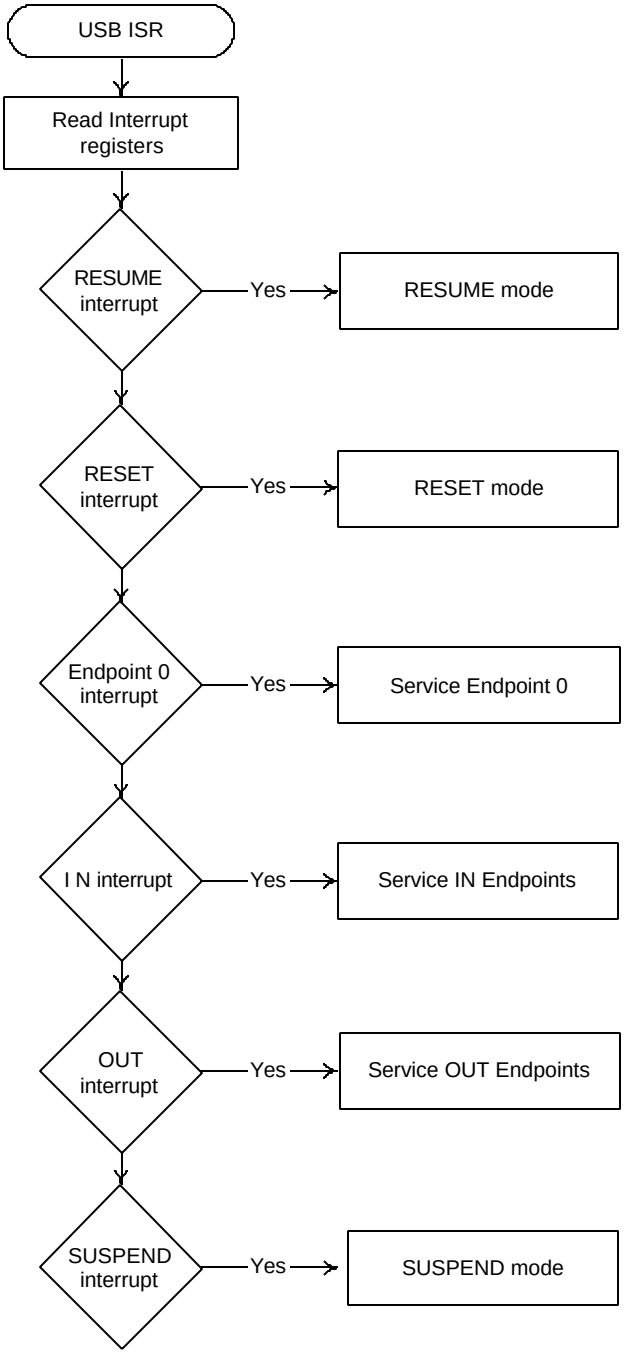
Throughout this discussion, the controlling device is assumed to be a microcontroller running some firmware but it could be a customized hard-wired logic block.

3.1. USB INTERRUPT HANDLING

When the CPU is interrupted with a USB interrupt, it needs to read the interrupt status register to determine which endpoint(s) have caused the interrupt and jump to the appropriate routine. If multiple endpoints have caused the interrupt, Endpoint 0 should be serviced first, followed by the other endpoints. The Suspend interrupt should be serviced last.

A flowchart for the USB Interrupt Service Routine is given in Figure 3-1.

Figure 3-1 USB Interrupt Service Routine



© 2000-2002 Mentor Graphics Corporation.

4. USB RESET

When a reset condition is detected on the USB, the MUSBFSFC performs the following actions:

- Sets FAddr to 0.
- Sets Index to 0.
- Flushes all endpoint FIFOs.
- Clears all control/status registers.
- Enables all interrupts, except Suspend.
- Generates a Reset interrupt.

When the software receives a Reset interrupt, it should close any open pipes and wait for bus enumeration to begin.

5. SUSPEND/RESUME

When the MUSBFSFC sees no activity on the USB for 3 ms, it will generate a Suspend interrupt (if enabled).

It is up to the software to decide what, if anything, to disable when the USB is in Suspend mode.

5.1. MUSBFSFC ACTIVE DURING SUSPEND

The USB may exit Suspend mode by sending Resume signaling on the bus.

If the MUSBFSFC is left active while the USB is in Suspend mode, the MUSBFSFC will monitor the USB for Resume signaling. When Resume signaling appears on the bus, the MUSBFSFC will generate a Resume interrupt.

5.2. MUSBFSFC INACTIVE DURING SUSPEND

When the Suspend interrupt described above is received, the software may disable the MUSBFSFC by stopping its clock (this must be done by some external means). However, because the MUSBFSFC is disabled, it will not then be able to detect Resume signaling on the USB. As a result, some external hardware will be needed to detect Resume signaling (by monitoring the DIM and DIP signals), so that the clock to the MUSBFSFC can be restarted.

5.3. REMOTE WAKEUP

If the MUSBFSFC is in Suspend mode and the software wants to initiate a remote wakeup, it should write to the Power register to set the Resume bit (D2) to 1. (If the clock to the MUSBFSFC has been stopped, it will need to be restarted before this write can occur.)

The software should leave this bit set for approximately 10 ms (minimum of 2 ms, a maximum of 15 ms) then reset it to 0. By this time the hub should have taken over driving Resume signaling on the USB.

Note: No Resume interrupt will be generated when the software initiates a remote wakeup.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

6. ENDPOINT 0 HANDLING

Endpoint 0 is the main control endpoint of the core. As such, the routines required to service Endpoint 0 are more complicated than those required to service other endpoints.

The software is required to handle all the Standard Device Requests that may be received via Endpoint 0. These are described in *Universal Serial Bus Specification*, Revision 2.0, Chapter 9. The protocol for these device requests involves different numbers and types of transaction per transfer. To accommodate this, the CPU needs to take a state machine approach to command decoding and handling.

The Standard Device Requests can be divided into three categories: *Zero Data Requests* (in which all the information is included in the command), W*rite Requests* (in which the command will be followed by additional data), and *Read Requests* (in which the device is required to send data back to the host).

This section looks at the sequence of events that the software must perform to process the different types of device request.

Note: The Setup packet associated with any Standard Device Request should include an 8-byte command. Any Setup packet containing a command field of anything other than 8 bytes will be automatically rejected by the MUSBFSFC core.

6.1. ZERO DATA REQUESTS

Zero data requests have all their information included in the 8-byte command and require no additional data to be transferred. Examples of zero data Standard Device Requests are: SET_FEATURE, CLEAR_FEATURE, SET_ADDRESS, SET_CONFIGURATION, SET_INTERFACE.

The sequence of events will begin, as with all requests, when the software receives an Endpoint 0 interrupt. The OutPktRdy bit (CSR0.D0) will also have been set. The 8-byte command should then be read from the Endpoint 0 FIFO, decoded and the appropriate action taken. For example if the command is SET_ADDRESS, the 7-bit address value contained in the command should be written to the FAddr register.

The CSR0 register should then be written to set the ServicedOutPktRdy bit (D6) (indicating that the command has been read from the FIFO) and to set the DataEnd bit (D3) (indicating that no further data is expected for this request).

When the host moves to the status stage of the request, a second Endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software: the second interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the CSR0 register should be written to set the ServicedOutPktRdy bit (D6) and to set the SendStall bit (D5). When the host moves to the status stage of the request, the MUSBFSFC will send a STALL to tell the host that the request was not executed. A second Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

If the host sends more data after the DataEnd bit has been set, then the MUSBFSFC will send a STALL. An Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

© 2000-2002 Mentor Graphics Corporation.

6.2. WRITE REQUESTS

Write requests involve an additional packet (or packets) of data being sent from the host after the 8-byte command. An example of a write Standard Device Request is: SET_DESCRIPTOR.

The sequence of events will begin, as with all requests, when the software receives an Endpoint 0 interrupt. The OutPktRdy bit (CSR0.D0) will also have been set. The 8-byte command should then be read from the Endpoint 0 FIFO and decoded.

As with a zero data request, the CSR0 register should then be written to set the ServicedOutPktRdy bit (D6) (indicating that the command has been read from the FIFO) but in this case the DataEnd bit (D3) should not be set (indicating that more data is expected).

When a second Endpoint 0 interrupt is received, the CSR0 register should be read to check the endpoint status. The OutPktRdy bit (CSR0.D0) should be set to indicate that a data packet has been received. The COUNT0 register should then be read to determine the size of this data packet. The data packet can then be read from the Endpoint 0 FIFO.

If the length of the data associated with the request (indicated by the *wLength* field in the command) is greater than the maximum packet size for Endpoint 0, further data packets will be sent. In this case, CSR0 should be written to set the ServicedOutPktRdy bit, but the DataEnd bit should not be set.

When all the expected data packets have been received, the CSR0 register should be written to set the ServicedOutPktRdy bit and to set the DataEnd bit (indicating that no more data is expected).

When the host moves to the status stage of the request, another Endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software, the interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the CSR0 register should be written to set the ServicedOutPktRdy bit (D6) and to set the SendStall bit (D5). When the host sends more data, the MUSBFSFC will send a STALL to tell the host that the request was not executed. An Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

If the host sends more data after the DataEnd has been set, then the MUSBFSFC will send a STALL. An Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

6.3. READ REQUESTS

Read requests have a packet (or packets) of data sent from the function to the host after the 8-byte command. Examples of read Standard Device Requests are: GET_CONFIGURATION, GET_INTERFACE, GET_DESCRIPTOR, GET_STATUS, SYNCH_FRAME.

The sequence of events will begin, as with all requests, when the software receives an Endpoint 0 interrupt. The OutPktRdy bit (CSR0.D0) will also have been set. The 8-byte command should then be read from the Endpoint 0 FIFO and decoded. The CSR0 register should then be written to set the ServicedOutPktRdy bit (D6) (indicating that the command has read from the FIFO).

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

The data to be sent to the host should then be written to the Endpoint 0 FIFO. If the data to be sent is greater than the maximum packet size for Endpoint 0, only the maximum packet size should be written to the FIFO. The CSR0 register should then be written to set the InPktRdy bit (D1) (indicating that there is a packet in the FIFO to be sent). When the packet has been sent to the host, another Endpoint 0 interrupt will be generated and the next data packet can be written to the FIFO.

When the last data packet has been written to the FIFO, the CSR0 register should be written to set the InPktRdy bit and to set the DataEnd bit (D3) (indicating that there is no more data after this packet).

When the host moves to the status stage of the request, another Endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software: the interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the CSR0 register should be written to set the ServicedOutPktRdy bit (D6) and to set the SendStall bit (D5). When the host requests data, the MUSBFSFC will send a STALL to tell the host that the request was not executed. An Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

If the host requests more data after DataEnd (D3) has been set, then the MUSBFSFC will send a STALL. An Endpoint 0 interrupt will be generated and the SentStall bit (CSR0.D2) will be set.

6.4. ENDPOINT 0 STATES

The Endpoint 0 control needs three modes – IDLE, TX and RX – corresponding to the different phases of the control transfer and the states Endpoint 0 enters for the different phases of the transfer (see Figure 6-1 below).

The default mode on power-up or reset should be IDLE.

OutPktRdy (CSR0.D0) becoming set when Endpoint 0 is in IDLE state indicates a new device request. Once the device request is unloaded from the FIFO, the MUSBFSFC decodes the descriptor to find whether there is a Data phase and, if so, the direction of the Data phase for the control transfer (in order to set the FIFO direction).

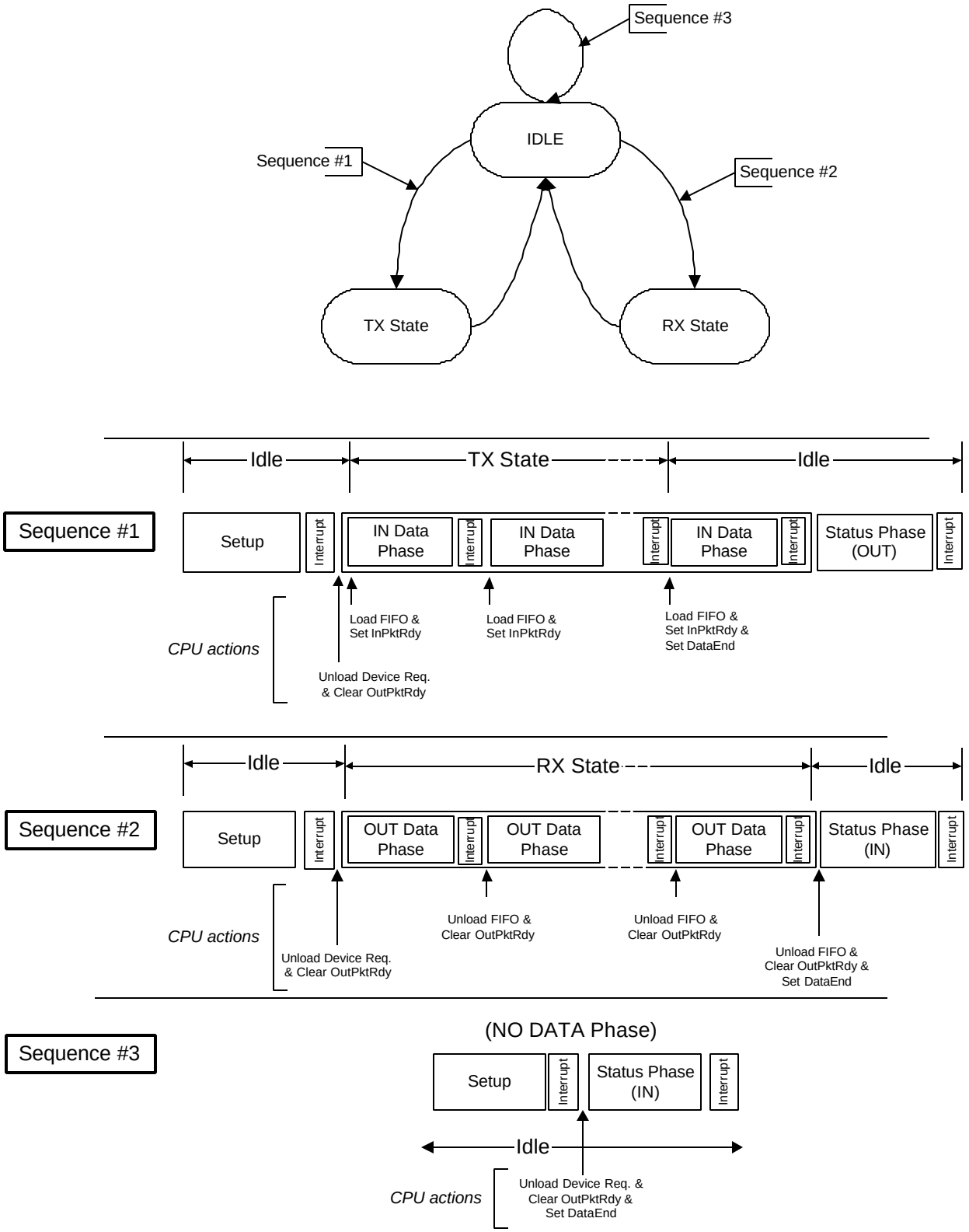
Depending on the direction of the Data phase, Endpoint 0 goes into either TX state or RX state. If there is no Data phase, Endpoint 0 remains in IDLE state to accept the next device request.

The actions that the CPU needs to take at the different phases of the possible transfers (e.g. Loading the FIFO, Setting InPktRdy) are indicated in the diagram on the following page.

Note that the MUSBFSFC changes the FIFO direction depending on the direction of the Data phase *independently* of the CPU.

© 2000-2002 Mentor Graphics Corporation.

Figure 6-1 Endpoint 0 States



CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

6.5. ENDPOINT 0 SERVICE ROUTINE

An Endpoint 0 interrupt is generated:

- When the core sets the OutPktRdy bit (CSR0.D0) after a valid token has been received and data has been written to the FIFO.
- When the core clears the InPktRdy bit (CSR0.D1) after the packet of data in the FIFO has been successfully transmitted to the host.
- When the core sets the SentStall bit (CSR0.D2) after a control transaction is ended due to a protocol violation.
- When the core sets the SetupEnd bit (CSR0.D4) because a control transfer has ended before DataEnd (CSR0.D3) is set.

Whenever the Endpoint 0 service routine is entered, the firmware must first check to see if the current control transfer has been ended due to either a STALL condition or a premature end of control transfer. If the control transfer ends due to a STALL condition, the SentStall bit would be set. If the control transfer ends due to a premature end of control transfer, the SetupEnd bit would be set. In either case, the firmware should abort processing the current control transfer and set the state to IDLE.

Once the firmware has determined that the interrupt was not generated by an illegal bus state, the next action taken depends on the Endpoint state.

If Endpoint 0 is in IDLE state, the only valid reason an interrupt can be generated is as a result of the core receiving data from the USB bus. The service routine must check for this by testing the OutPktRdy (CSR0.D0) bit. If this bit is set, then the core has received a SETUP packet. This must be unloaded from the FIFO and decoded to determine the action the core must take. Depending on the command contained within the SETUP packet, Endpoint 0 will enter one of three states:

- If the command is a single packet transaction (SET_ADDRESS, SET_INTERFACE etc.) without any data phase, the endpoint will remain in IDLE state.
- If the command has an OUT data phase (SET_DESCRIPTOR etc.), the endpoint will enter RX state.
- If the command has an IN data phase (GET_DESCRIPTOR etc.), the endpoint will enter TX state.

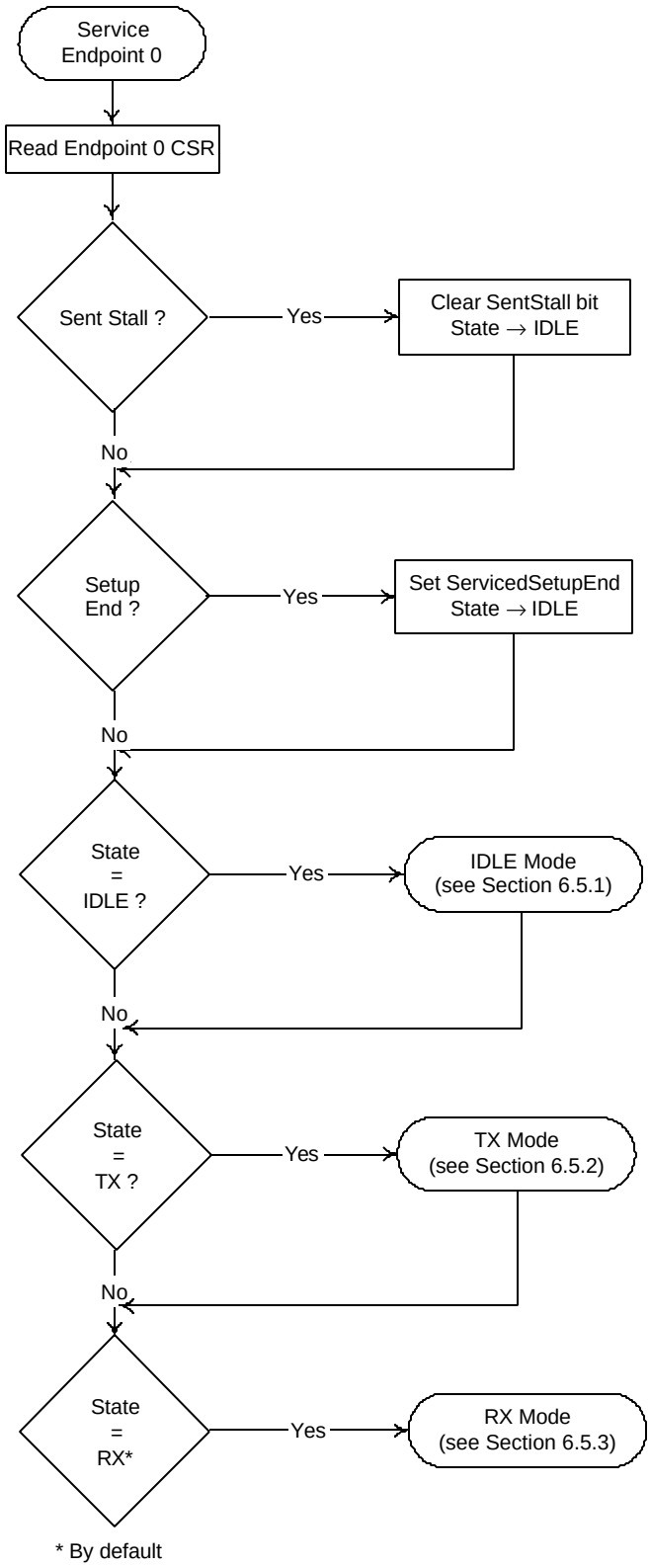
If the endpoint is in TX state, the interrupt indicates that the core has received an IN token and data from the FIFO has been sent. The firmware must respond to this either by placing more data in the FIFO if the host is still expecting more data[2] or by setting the DataEnd bit to indicate that the data phase is complete. Once the data phase of the transaction has been completed, Endpoint 0 should be returned to IDLE state to await the next control transaction.

If the endpoint is in RX state, the interrupt indicates that a data packet has been received. The firmware must respond by unloading the received data from the FIFO. The firmware must then determine whether it has received all of the expected data[2]. If it has, the firmware should set the DataEnd bit and return Endpoint 0 to IDLE state. If more data is expected, the firmware should set the ServicedOutPktRdy bit (CSR0.D6) to indicate that it has read the data in the FIFO and leave the endpoint in RX state.

[2] Command transactions all include a field that indicates the amount of data the host expects to receive or is going to send.

© 2000-2002 Mentor Graphics Corporation.

Figure 6-2 Endpoint 0 Service Routine



CONFIDENTIAL

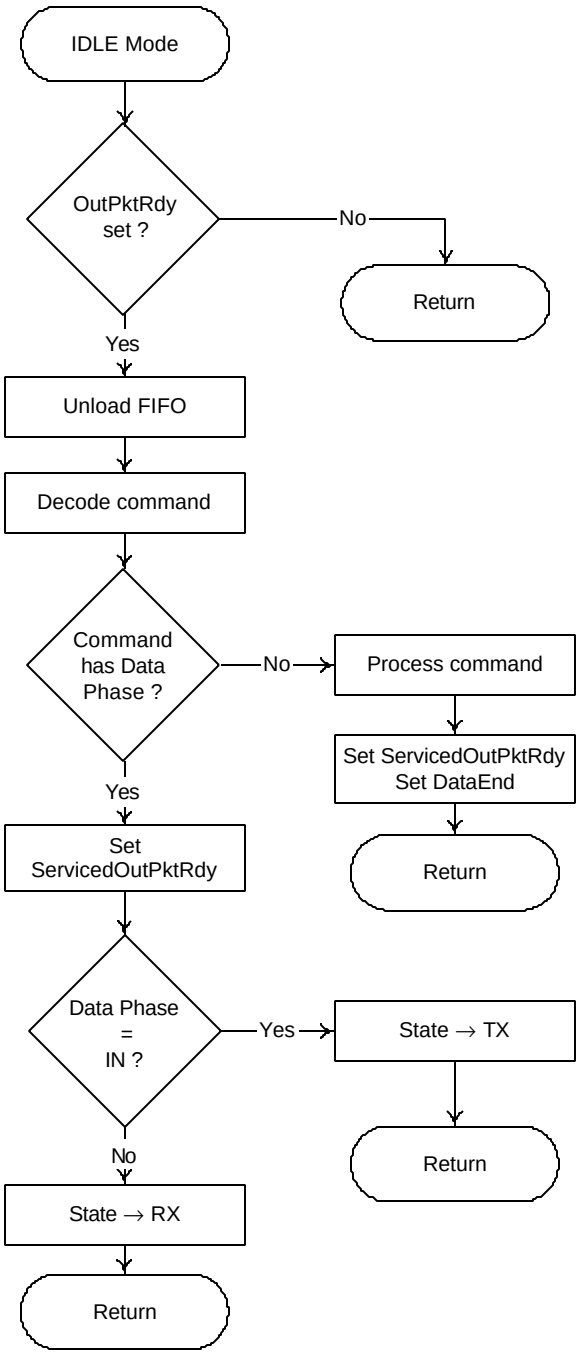
© 2000-2002 Mentor Graphics Corporation.

6.5.1. IDLE MODE

IDLE mode is the mode the Endpoint 0 control needs to select at power-on or reset and is the mode to which the Endpoint 0 control should return when the RX and TX modes are terminated.

It is also the mode in which the SETUP phase of control transfer is handled (as outlined in Figure 6-3 below).

Figure 6-3 SETUP Phase of Control Transfer



© 2000-2002 Mentor Graphics Corporation.

6.5.2. TX MODE

When the endpoint is in TX state, all arriving IN tokens need to be treated as part of a Data phase until the required amount of data has been sent to the host. If either a SETUP or an OUT token is received whilst the endpoint is in the TX state, this will cause a SetupEnd condition to occur as the core expects only IN tokens.

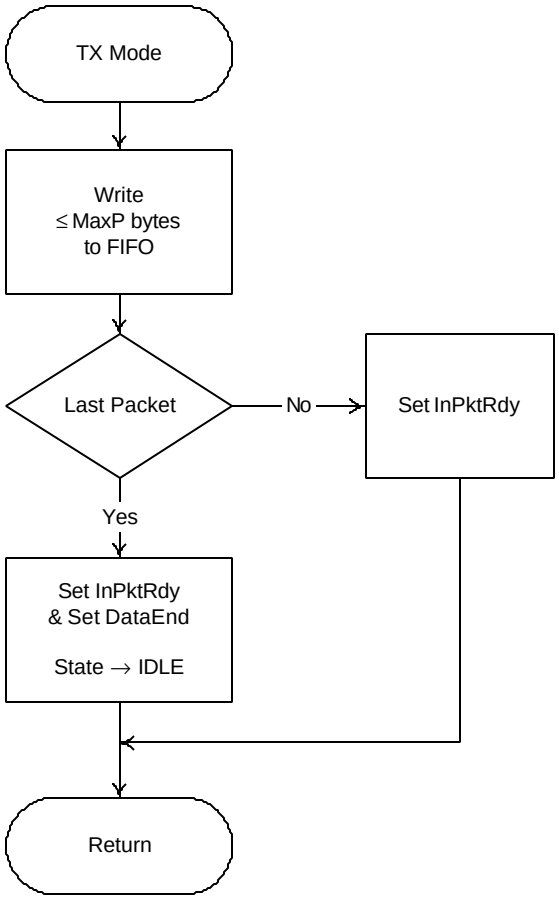
Three events can cause TX mode to be terminated before the expected amount of data has been sent:

- The host sends an invalid token causing a SetupEnd condition (CSR0.D4 set)
- The firmware sends a packet containing less than the maximum packet size for Endpoint 0 (MaxP)
- The firmware sends an empty data packet

Until the transaction is terminated, the firmware simply needs to load the FIFO when it receives an interrupt which indicates that a packet has been sent from the FIFO. (An interrupt is generated when InPktRdy is cleared.)

When the firmware forces the termination of a transfer (by sending a short or empty data packet), it should set the DataEnd bit (CSR0.D3) to indicate to the core that the Data phase is complete and that the core should next receive an acknowledge packet.

Figure 6-4 IN Data Phase for Control transfer



CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

6.5.3. RX MODE

In RX mode, all arriving data should be treated as part of a Data phase until the expected amount of data has been received. If either a SETUP or an IN token is received while the endpoint is in RX state, this will cause a SetupEnd condition to occur as the core expects only OUT tokens.

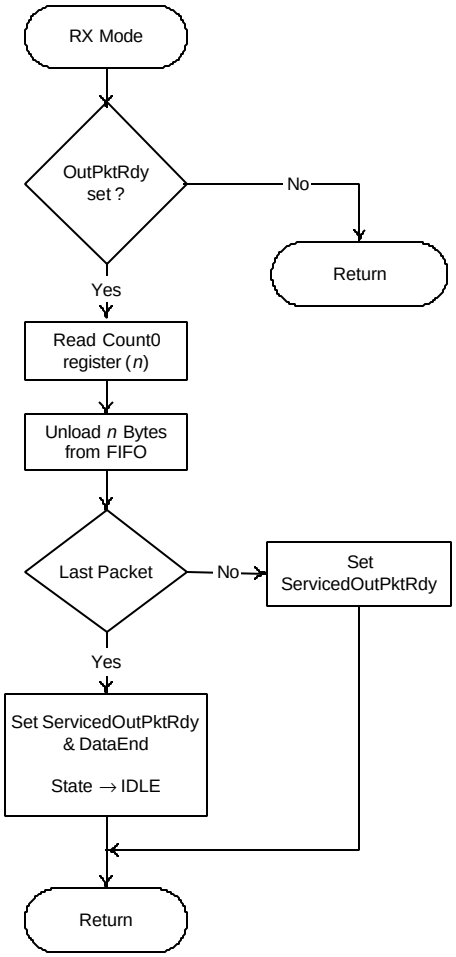
Three events can cause RX mode to be terminated before the expected amount of data has been received:

- The host sends an invalid token causing a SetupEnd condition (CSR0.D4 set)
- The host sends a packet which contains less than the maximum packet size for Endpoint 0
- The host sends an empty data packet

Until the transaction is terminated, the firmware simply needs to unload the FIFO when it receives an interrupt which indicates that new data has arrived (OutPktRdy (CSR0.D0) set) and to clear OutPktRdy by setting the ServicedOutPktRdy bit (CSR0.D6).

When the firmware detects the termination of a transfer (by receiving either the expected amount of data or an empty data packet), it should set the DataEnd bit (CSR0.D3) to indicate to the core that the Data phase is complete and that the core should receive an acknowledge packet next.

Figure 6-5 OUT Data Phase for Control Transfer



CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

6.6. ERROR HANDLING

A control transfer may be aborted due to a protocol error on the USB, the host prematurely ending the transfer, or if the function controller software wishes to abort the transfer (e.g. because it cannot process the command).

The MUSBFSFC will automatically detect protocol errors and send a STALL packet to the host under the following conditions:

1. The host sends more data during the OUT Data phase of a write request than was specified in the command. This condition is detected when the host sends an OUT token after the DataEnd bit (CSR0.D3) has been set.

2. The host request more data during the IN Data phase of a read request than was specified in the command. This condition is detected when the host sends an IN token after the DataEnd bit in the CSR0 register has been set.

3. The host sends more than MaxP data bytes in an OUT data packet.

4. The host sends a non-zero length DATA1 packet during the STATUS phase of a read request.

When the MUSBFSFC has sent the STALL packet, it sets the SentStall bit (CSR0.D2) and generates an interrupt. When the software receives an Endpoint 0 interrupt with the SentStall bit set, it should abort the current transfer, clear the SentStall bit, and return to the IDLE state.

If the host prematurely ends a transfer by entering the STATUS phase before all the data for the request has been transferred, or by sending a new SETUP packet before completing the current transfer, then the SetupEnd bit (CSR0.D4) will be set and an Endpoint 0 interrupt generated. When the software receives an Endpoint 0 interrupt with the SetupEnd bit set, it should abort the current transfer, set the ServicedSetupEnd bit (CSR0.D7), and return to the IDLE state. If the OutPktRdy bit (CSR0.D0) is set this indicates that the host has sent another SETUP packet and the software should then process this command.

If the software wants to abort the current transfer, because it cannot process the command or has some other internal error, then it should set the SendStall bit (CSR0.D5). The MUSBFSFC will then send a STALL packet to the host, set the SentStall bit (CSR0.D2) and generate an Endpoint 0 interrupt.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

7. BULK IN ENDPOINT

A Bulk IN endpoint is used to transfer non-periodic data from the function controller to the host.

Three optional features are available for use with a Bulk IN endpoint:

- **Double packet buffering**

If the value written to the InMaxP register is less than, or equal to, half the size of the FIFO allocated to the endpoint, double packet buffering will be automatically enabled. When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host.

- **DMA**

If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature can be used to allow an external DMA controller to load packets into the FIFO without processor intervention.

- **AutoSet**

When the AutoSet feature is enabled, the InPktRdy bit (InCSR1.D0) will be automatically set when a packet of InMaxP bytes has been loaded into the FIFO. This is particularly useful when DMA is used to load the FIFO as it avoids the need for any processor intervention when loading individual packets during a large Bulk transfer.

7.1. SETUP

Before using a Bulk IN endpoint, the InMaxP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the IntrIn1E/InIntr2E register should be set to 1 (if an interrupt is required for this endpoint) and the InCSR2 register should be set as shown below (Bits D2 – D0 of InCSR2 are unused):

D7	**AutoSet**	0/1	Set to 1 if the AutoSet feature is required.
D6	**ISO**	0	Set to 0 to enable Bulk protocol.
D5	**Mode**	1	Set to 1 to ensure the FIFO is enabled (only necessary if the FIFO is shared with an OUT endpoint).
D4	**DMAEnab**	0/1	Set to 1 if a DMA request is required for this endpoint.
D3	**FrcDataTog**	0	Set to 0 to allow normal data toggle operation.

When a Bulk IN endpoint is first configured, following a SET_CONFIGURATION or SET_INTERFACE command on Endpoint 0, the InCSR1 register should be written to set the ClrDataTog bit (D6). This will ensure that the data toggle (which is handled automatically by the MUSBFSFC) starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the FIFONotEmpty bit (InCSR1.D1) being set), they should be flushed by setting the FlushFIFO bit (InCSR1.D3). *Note:* It may be necessary to set this bit twice in succession if double buffering is enabled.

© 2000-2002 Mentor Graphics Corporation.

7.2. OPERATION

When data is to be transferred over a Bulk IN pipe, a data packet needs to be loaded into the FIFO and the InCSR1 register written to set the InPktRdy bit (D0). When the packet has been sent, the InPktRdy bit will be cleared by the MUSBFSFC and an interrupt generated so that the next packet can be loaded into the FIFO. If double packet buffering is enabled, then after the first packet has been loaded and the InPktRdy bit set, the InPktRdy bit will immediately be cleared by the MUSBFSFC and an interrupt generated so that a second packet can be loaded into the FIFO. The software should operate in the same way, loading a packet when it receives an interrupt, regardless of whether double packet buffering is enabled or not.

The packet size must not exceed the size specified in the InMaxP register. When a block of data larger than InMaxP is to be transferred, it must be sent as multiple packets. These packets should be InMaxP in size, except the last packet which holds the residue. The host may determine that all the data for a transfer has been sent by knowing the total amount of data that is expected. Alternatively it may infer that all the data have been sent when it receives a packet which is less than InMaxP in size. In the latter case, if the total size of the data block is a multiple of InMaxP, it will be necessary for the function to send a null packet after all the data has been sent. This is done by setting InPktRdy when the next interrupt is received, without loading any data into the FIFO.

If large blocks of data are being transferred, then the overhead of calling an interrupt service routine to load each packet can be avoided by using DMA. See Section 9.1 of the MUSBFSFC User Guide.

7.3. ERROR HANDLING

If the software wants to shut down the Bulk IN pipe, it should set the SendStall bit (InCSR1.D4). When the MUSBFSFC receives the next IN token, it will send a STALL to the host, set the SentStall bit (InCSR1.D5) and generate an interrupt.

When the software receives an interrupt with the SentStall bit (InCSR1.D5) set, it should clear the SentStall bit. It should leave the SendStall bit (InCSR1.D4) set until it is ready to re-enable the Bulk IN pipe. *Note:* If the host failed to receive the STALL packet for some reason, it will send another IN token, so it is advisable to leave the SendStall bit set until the software is ready to re-enable the Bulk IN pipe. When a pipe is re-enabled, the data toggle sequence should be restarted by setting the ClrDataTog bit in the InCSR1 register (D6).

8. BULK OUT ENDPOINT

A Bulk OUT endpoint is used to transfer non-periodic data from the host to the function controller.

Three optional features are available for use with a Bulk OUT endpoint:

- **Double packet buffering**

If the value written to the OutMaxP register is less than, or equal to, half the size of the FIFO allocated to the endpoint, double packet buffering will be automatically enabled. When enabled, up to two packets can be stored in the FIFO.

- **DMA**

If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow an external DMA controller to unload packets from the FIFO without processor intervention.

- **AutoClear**

When the AutoClear feature is enabled, the OutPktRdy bit (OutCSR1.D0) will be automatically cleared when a packet of OutMaxP bytes has been unloaded from the FIFO. This is particularly useful when DMA is used to unload the FIFO as it avoids the need for any processor intervention when unloading individual packets during a large Bulk transfer.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

8.1. SETUP

Before using a Bulk OUT endpoint, the OutMaxP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the IntrOut1E/IntrOut2E register should be set to 1 (if an interrupt is required for this endpoint) and the OutCSR2 register should be set as shown below (Bits D4 – D0 of OutCSR2 are unused.):

D7	**AutoClear**	0/1	Set to 1 if the AutoClear feature is required.
D6	**ISO**	0	Set to 0 to enable Bulk protocol.
D5	**DMAEnab**	0/1	Set to 1 if a DMA request is required for this endpoint.

When a Bulk OUT endpoint is first configured, following a SET_CONFIGURATION or SET_INTERFACE command on Endpoint 0, OutCSR1 should be written to set the ClrDataTog bit (D7). This will ensure that the data toggle (which is handled automatically by the MUSBFSFC) starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the OutPktRdy bit (OutCSR1.D0) being set), they should be flushed by setting the FlushFIFO bit (OutCSR1.D4). *Note:* It may be necessary to set this bit twice in succession if double buffering is enabled.

8.2. OPERATION

When a data packet is received by a Bulk OUT endpoint, the OutPktRdy bit (OutCSR1.D0) is set and an interrupt is generated. The software should read the two OutCount registers for the endpoint to determine the size of the data packet. The data packet should be read from the FIFO, then the OutPktRdy bit should be cleared.

The packet sizes should not exceed the size specified in the OutMaxP register (as this should be the value set in the *wMaxPacketSize* field of the endpoint descriptor sent to the host). When a block of data larger than OutMaxP is to be sent to the function, it will be sent as multiple packets. All the packets will be OutMaxP in size, except the last packet which will contain the residue. The software may use an application specific method of determining the total size of the block and hence when the last packet has been received. Alternatively it may infer that the entire block has been received when it receives a packet which is less than OutMaxP in size. (If the total size of the data block is a multiple of OutMaxP, a null data packet will be sent after the data to signify that the transfer is complete.)

If large blocks of data are being transferred, the overhead of calling an interrupt service routine to unload each packet can be avoided by using DMA. See Section 9.2 of the MUSBFSFC User Guide.

8.3. ERROR HANDLING

If the software wants to shut down the Bulk OUT pipe, it should set the SendStall bit (OutCSR1.D5). When the MUSBFSFC receives the next packet it will send a STALL to the host, set the SentStall bit (OutCSR1.D6) and generate an interrupt.

When the software receives an interrupt with the SentStall bit (OutCSR1.D6) set, it should clear the SentStall bit. It should leave the SendStall bit (OutCSR1.D5) set until it is ready to re-enable the Bulk OUT pipe. *Note:* If the host failed to receive the STALL packet for some reason, it will send another packet, so it is advisable to leave the SendStall bit set until the software is ready to re-enable the Bulk OUT pipe. When a Bulk OUT pipe is re-enabled, the data toggle sequence should be restarted by setting the ClrDataTog bit in the OutCSR1 register (D7).

© 2000-2002 Mentor Graphics Corporation.

9. INTERRUPT IN ENDPOINT

An Interrupt IN endpoint is used to transfer periodic data from the function controller to the host.

An Interrupt IN endpoint uses the same protocol as a Bulk IN endpoint and can be used the same way. However, though DMA can be used, it offers little benefit as Interrupt endpoints are usually expected to transfer all their data in a single packet.

Interrupt IN endpoints also support one feature that Bulk IN endpoints do not, in that they support continuous toggle of the data toggle bit. This feature is enabled by setting the FrcDataTog bit in the InCSR2 register (D3). When this bit is set to 1, the MUSBFSFC will consider the packet as having been successfully sent and toggle the data bit for the endpoint, regardless of whether an ACK was received from the host.

10. INTERRUPT OUT ENDPOINT

An Interrupt OUT endpoint is used to transfer periodic data from the host to a function controller.

An Interrupt OUT endpoint uses almost the same protocol as a Bulk OUT endpoint and can be used the same way. However, although DMA can be used with an Interrupt OUT endpoint, it generally offers little benefit as Interrupt endpoints are usually expected to transfer all their data in a single packet.

11. ISOCHRONOUS IN ENDPOINT

An Isochronous IN endpoint is used to transfer periodic data from the function controller to the host.

Three optional features are available for use with an Isochronous IN endpoint:

- **Double packet buffering**

Double packet buffering is automatically enabled when the value written to the InMaxP register is less than or equal to half the size of the FIFO allocated to the endpoint. When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host. *Note:* Double packet buffering is generally advisable for Isochronous IN endpoints in order to avoid data underrun (see 'Operation' below).

- **DMA**

If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature can be used to allow an external DMA controller to load packets into the FIFO without processor intervention. However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the InCSR1 register needs to be accessed following every packet to check for Underrun errors.

- **AutoSet**

When the AutoSet feature is enabled, the InPktRdy bit (InCSR1.D0) will be automatically set when a packet of InMaxP bytes has been loaded into the FIFO. However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the InCSR1 register needs to be accessed following every packet to check for Underrun errors.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

11.1. SETUP

Before using an Isochronous IN endpoint, the InMaxP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the IntrIn1E/Intr2E register should be set to 1 (if an interrupt is required for this endpoint) and the InCSR2 register should be set as shown below (Bits D2 – D0 of InCSR2 are unused.).

D7	**AutoSet**	0/1	Set to 1 if the AutoSet feature is required.
D6	**ISO**	1	Set to 1 to enable Isochronous protocol.
D5	**Mode**	1	Set to 1 to ensure the FIFO is enabled (only necessary if the FIFO is shared with an OUT endpoint).
D4	**DMAEnab**	0/1	Set to 1 if a DMA request is required for this endpoint.
D3	**FrcDataTog**	0	Ignored in Isochronous mode.

11.2. OPERATION

An Isochronous endpoint does not support data retries, so if data underrun is to be avoided, the data to be sent to the host must be loaded into the FIFO before the IN token is received. The host will send one IN token per frame, however the time within the frame can vary. If an IN token is received near the end of one frame and then at the start of the next frame, there will be little time to reload the FIFO. For this reason, double buffering is usually required for an Isochronous IN endpoint.

The AutoSet feature can be used with an Isochronous IN endpoint, in the same way as for a Bulk IN endpoint. However, unless the data arrives from the source at an absolutely consistent rate, synchronized to the host's frame clock, the size of the packets sent to the host will have to increase or decrease from frame to frame to match the source data rate. This means that the actual packet sizes will not always be InMaxP in size, rendering the AutoSet feature useless.

An interrupt is generated whenever a packet is sent to the host and the software may use this interrupt to load the next packet into the FIFO and set the InPktRdy bit in the InCSR1 register (D0) in the same way as for a Bulk IN endpoint. As the interrupt could occur almost any time within a frame, depending on when the host has scheduled the transaction, this may result in irregular timing of FIFO load requests. If the data source for the endpoint is coming from some external hardware, it may be more convenient to wait until the end of each frame before loading the FIFO as this will minimize the requirement for additional buffering. This can be done by using either the SOF interrupt (IntrUSB.D3) or the external SOF_PULSE signal from the MUSBFSFC to trigger the loading of the next data packet. The SOF_PULSE is generated once per frame when a SOF packet is received. (The MUSBFSFC also maintains an external frame counter so it can still generate a SOF_PULSE when the SOF packet has been lost.) The interrupts may still be used to set the InPktRdy bit in InCSR1 (D0) and to check for data overruns/underruns (see 'Error Handling' below).

Starting up a double-buffered Isochronous IN pipe can be a source of problems. Double buffering requires that a data packet is not transmitted until the frame after it is loaded. There is no problem if the function loads the first data packet at least a frame before the host sets up the pipe (and therefore starts sending IN tokens). But if the host has already started sending IN tokens by the time the first packet is loaded, the packet may be transmitted in the same frame as it is loaded, depending on whether it is loaded before, or after, the IN token is received. This potential problem can be avoided by setting the ISO Update bit in the Power register (D7). When this bit is set to 1, any data packet loaded into an Isochronous IN endpoint FIFO will not be transmitted until after the next SOF packet has been received, thereby ensuring that the data packet is not sent too early.

© 2000-2002 Mentor Graphics Corporation.

11.3. ERROR HANDLING

If the endpoint has no data in its FIFO when an IN token is received, it will send a null data packet to the host and set the UnderRun bit in the InCSR1 register (D2). This is an indication that the software is not supplying data fast enough for the host.
It is up to the application to determine how this error condition is handled.

If the software is loading one packet per frame and it finds that the InPktRdy bit in the InCSR1 register (D0) is set when it wants to load the next packet, this indicates that a data packet has not been sent (perhaps because an IN token from the host was corrupted). It is up to the application how it handles this condition: it may choose to flush the unsent packet by setting the FlushFIFO bit in the InCSR1 register (D3), or it may choose to skip the current packet.

12. ISOCHRONOUS OUT ENDPOINT

An Isochronous OUT endpoint is used to transfer periodic data from the host to the function controller.

Three optional features are available for use with an Isochronous OUT endpoint:

- **Double packet buffering**

Double packet buffering is automatically enabled when the value written to the OutMaxP register is less than or equal to half the size of the FIFO allocated to the endpoint. When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host. *Note:* Double packet buffering is generally advisable for Isochronous OUT endpoints in order to avoid data overrun (see 'Operation' below).

- **DMA**

If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow an external DMA controller to unload packets from the FIFO without processor intervention. However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the OutCSR1 register needs to be accessed following every packet to check for Overrun or CRC errors.

- **AutoClear**

When the AutoClear feature is enabled, the OutPktRdy bit (OutCSR1.D0) will be automatically cleared when a packet of OutMaxP bytes has been unloaded from the FIFO. However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the OutCSR1 register needs to be accessed following every packet to check for Overrun or CRC errors.

12.1. SETUP

Before using an Isochronous OUT endpoint, the OutMaxP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the *wMaxPacketSize* field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the IntrOut1E/IntrOut2E register should be set to 1 (if an interrupt is required for this endpoint) and the OutCSR2 register should be set as shown below (Bits D4 – D0 of OutCSR2 are unused.):

D7	**AutoClear**	0/1	Set to 1 if the AutoClear feature is required.
D6	**ISO**	1	Set to 1 to enable Isochronous protocol.
D5	**DMAEnab**	0/1	Set to 1 if a DMA request is required for this endpoint.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

12.2. OPERATION

An Isochronous endpoint does not support data retries so, if a data overrun is to be avoided, there must be space in the FIFO to accept a packet when it is received. The host will send one packet per frame, however the time within the frame can vary. If a packet is received near the end of one frame and another arrives at the start of the next frame, there will be little time to unload the FIFO. For this reason, double buffering is usually required for an Isochronous OUT endpoint.

The AutoClear feature can be used with an Isochronous OUT endpoint, in the same way as for a Bulk OUT endpoint. However, unless the data sink receives data at an absolutely consistent rate and is synchronized to the host's frame clock, the size of the packets sent from the host will have to increase or decrease from frame to frame to match the required data rate. This means that the actual packet sizes will not always be OutMaxP in size, rendering the AutoClear feature useless.

An interrupt is generated whenever a packet is received from the host and the software may use this interrupt to unload the packet from the FIFO and clear the OutPktRdy bit in the OutCSR1 register (D0) in the same way as for a Bulk OUT endpoint. As the interrupt could occur almost any time within a frame, depending on when the host has scheduled the transaction, the timing of FIFO unload requests will probably be irregular. If the data sink for the endpoint is going to some external hardware, it may be better to minimize the requirement for additional buffering by waiting until the end of each frame before unloading the FIFO. This can be done by using either the SOF interrupt (IntrUSB.D3) or the external SOF_PULSE signal from the MUSBFSFC to trigger the unloading of the data packet. The SOF_PULSE is generated once per frame when a SOF packet is received. (The MUSBFSFC also maintains an external frame counter so it can still generate a SOF_PULSE when the SOF packet has been lost.) The interrupts may still be used to clear the OutPktRdy bit in OutCSR1 and to check for data overruns/underruns (see 'Error Handling' below).

12.3. ERROR HANDLING

If there is no space in the FIFO to store a packet when it is received from the host, the OverRun bit in the OutCSR1 register (D2) will be set. This is an indication that the software is not unloading data fast enough for the host. It is up to the application to determine how this error condition is handled.

If the MUSBFSFC finds that a received packet has a CRC error, it will still store the packet in the FIFO and set the OutPktRdy bit (OutCSR1.D0) and the DataError bit (OutCSR1.D3). It is left up to the application how this error condition is handled.

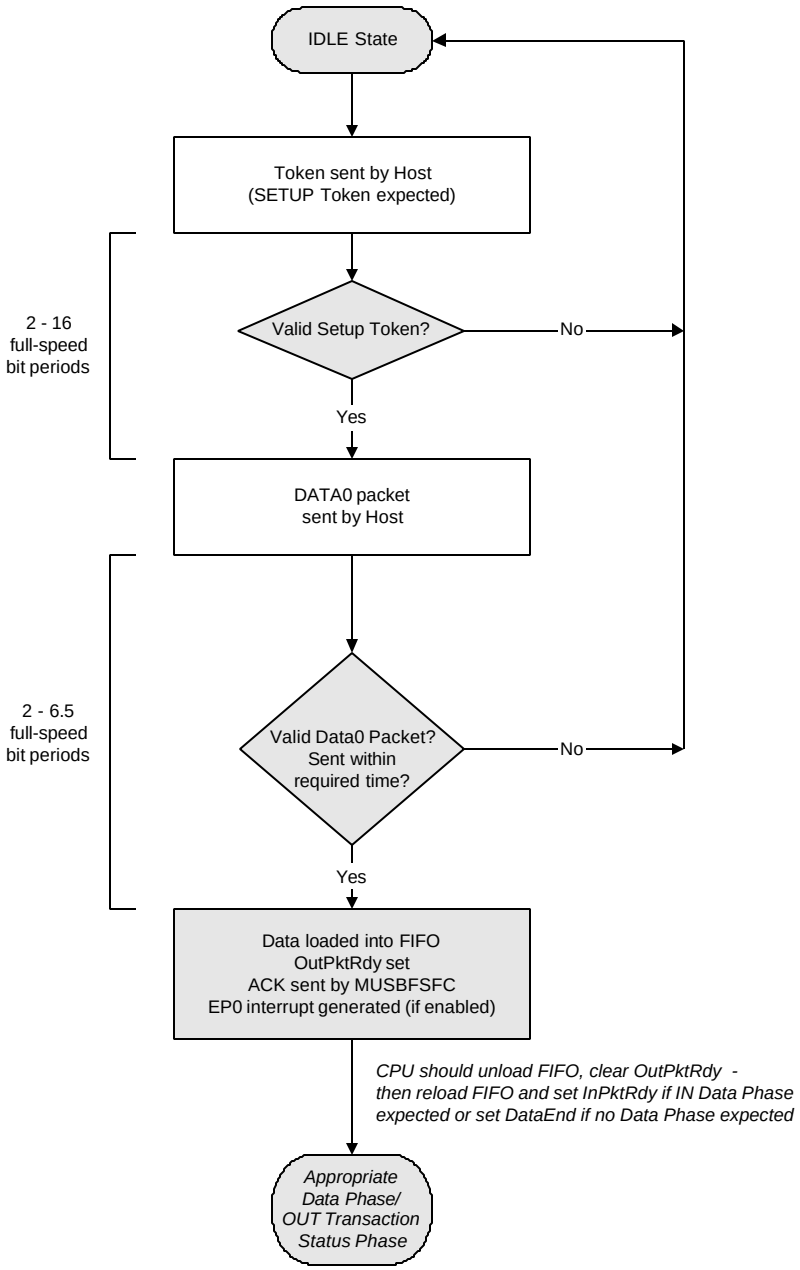
© 2000-2002 Mentor Graphics Corporation.

13. TRANSACTION FLOWS

Note: Host actions are shown against a white background. MUSBFSFC actions are shown shaded.

13.1. CONTROL TRANSACTIONS

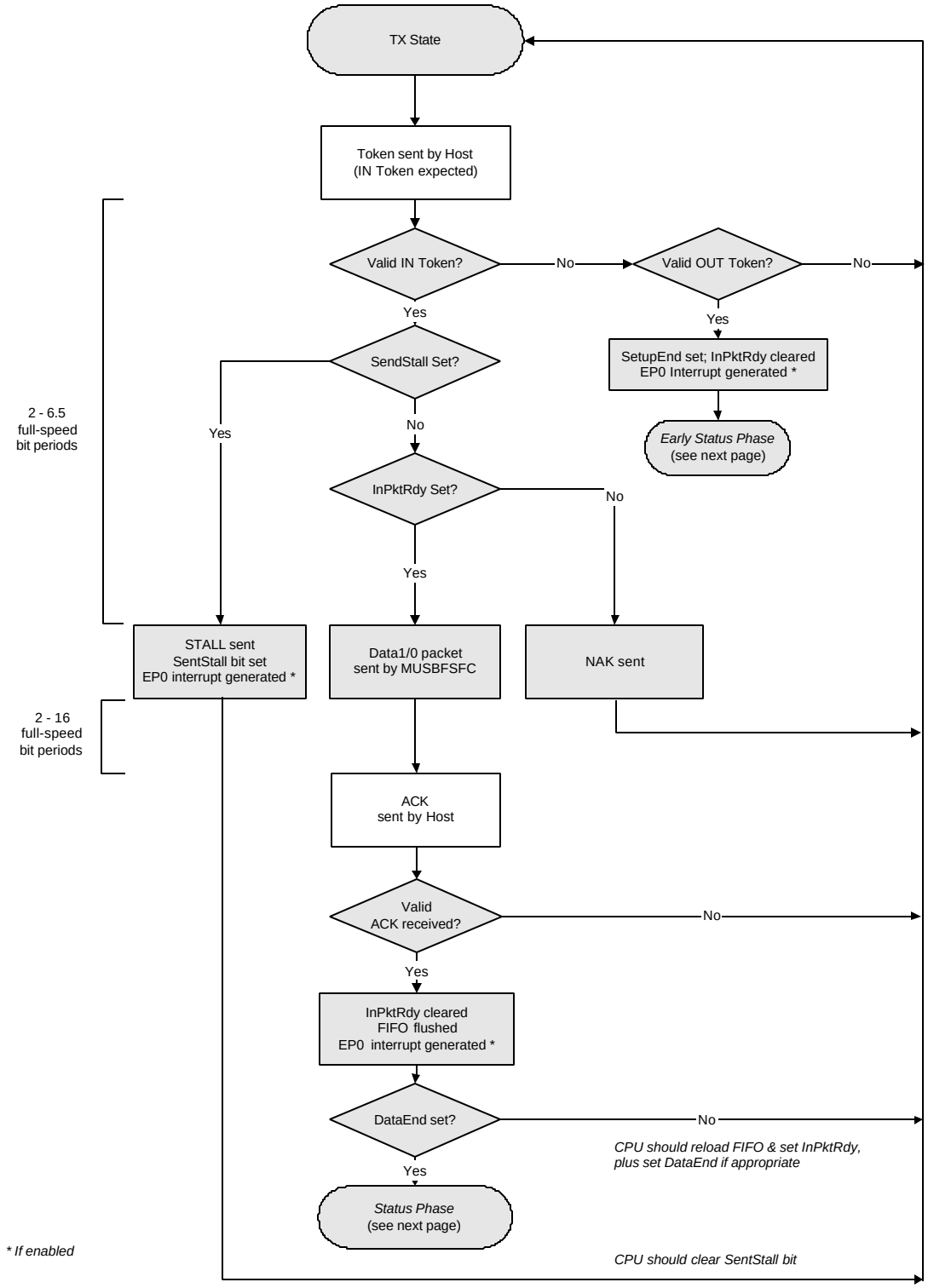
13.1.1. SETUP PHASE



CONFIDENTIAL

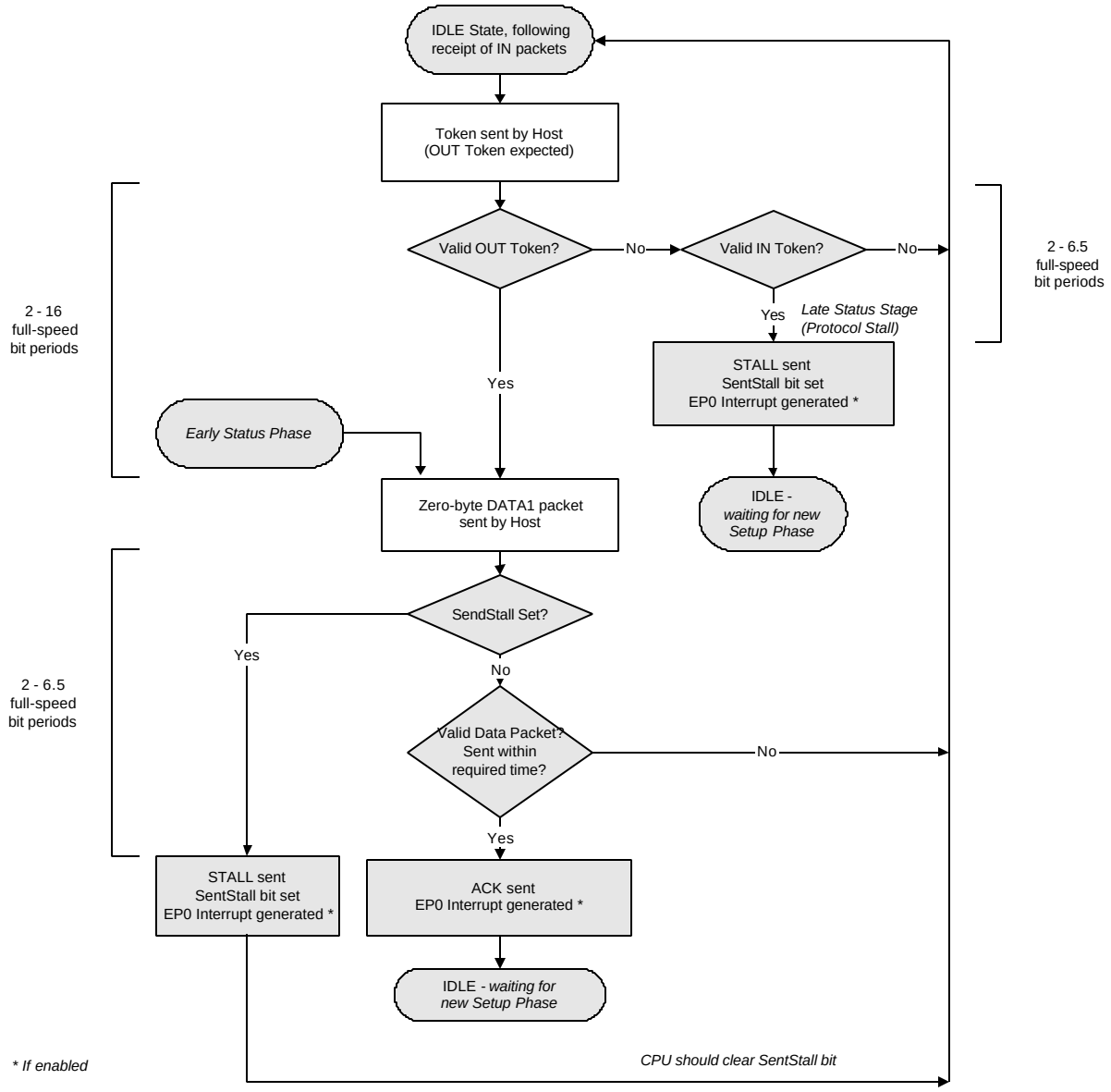
© 2000-2002 Mentor Graphics Corporation.

13.1.2. IN DATA PHASE ...



© 2000-2002 Mentor Graphics Corporation.

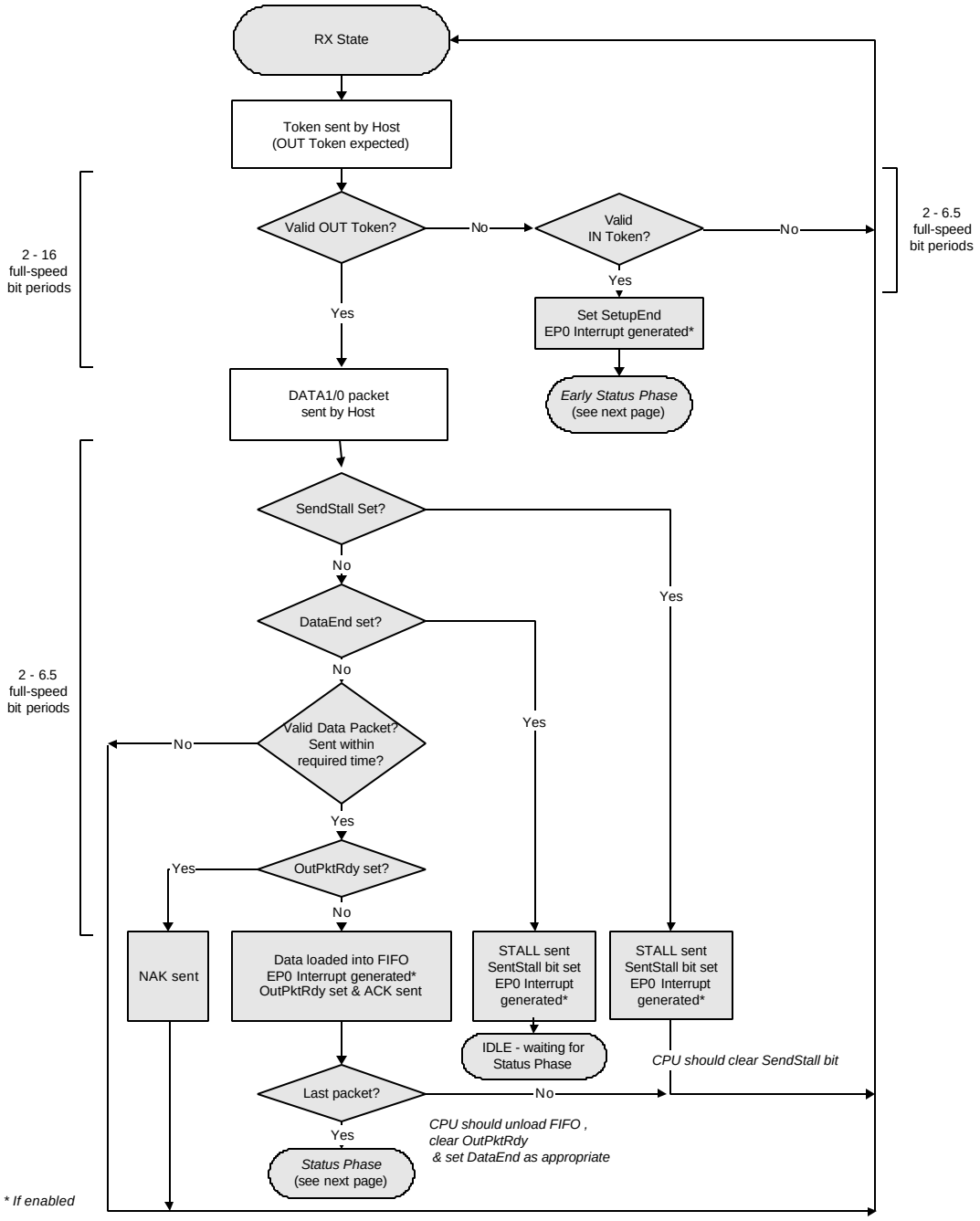
... AND FOLLOWING STATUS PHASE



CONFIDENTIAL

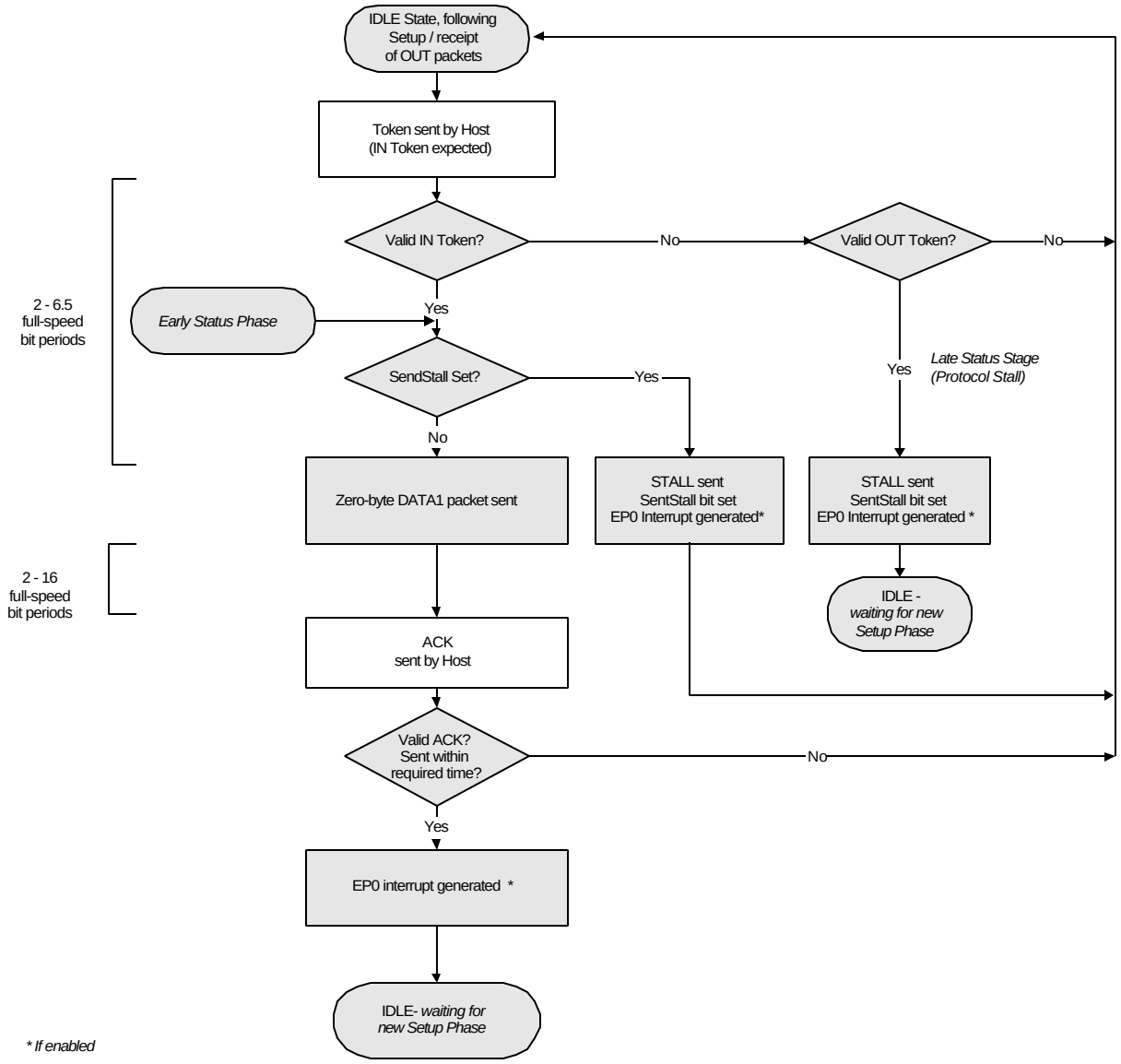
© 2000-2002 Mentor Graphics Corporation.

13.1.3. OUT DATA PHASE...



© 2000-2002 Mentor Graphics Corporation.

... AND FOLLOWING STATUS PHASE

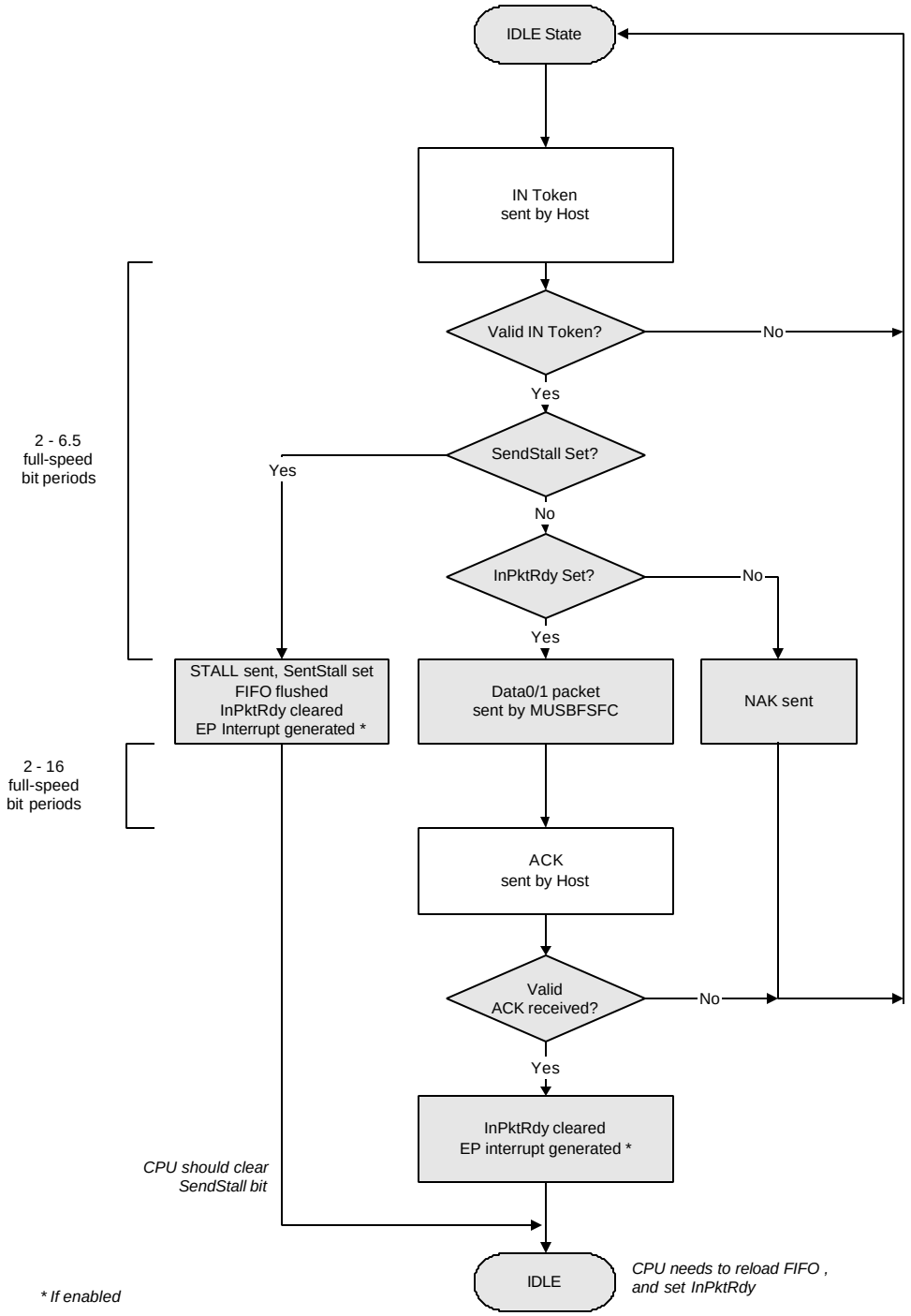


CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

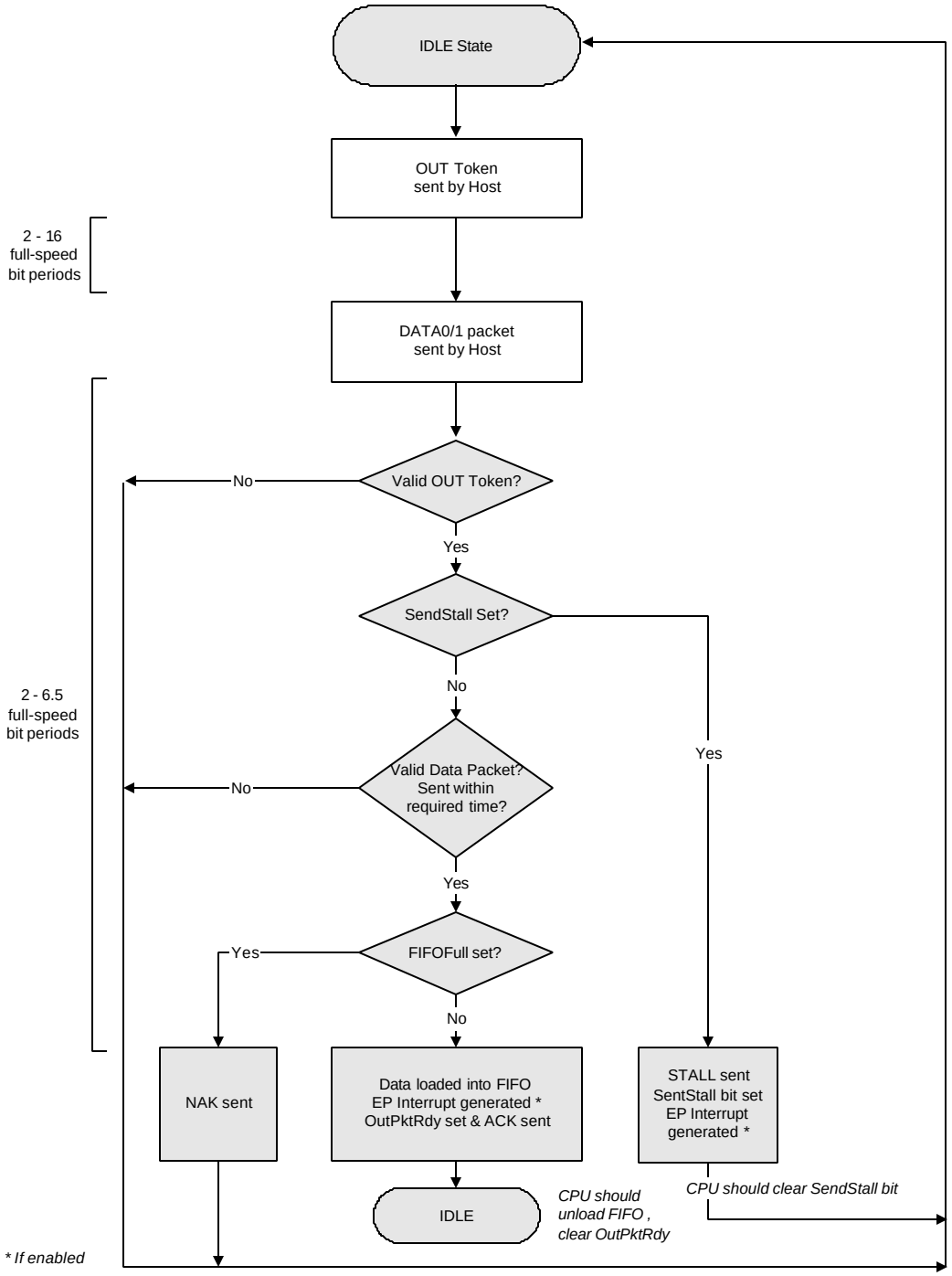
13.2. BULK/INTERRUPT TRANSACTIONS

13.2.1. IN TRANSACTION



© 2000-2002 Mentor Graphics Corporation.

13.2.2. OUT TRANSACTION

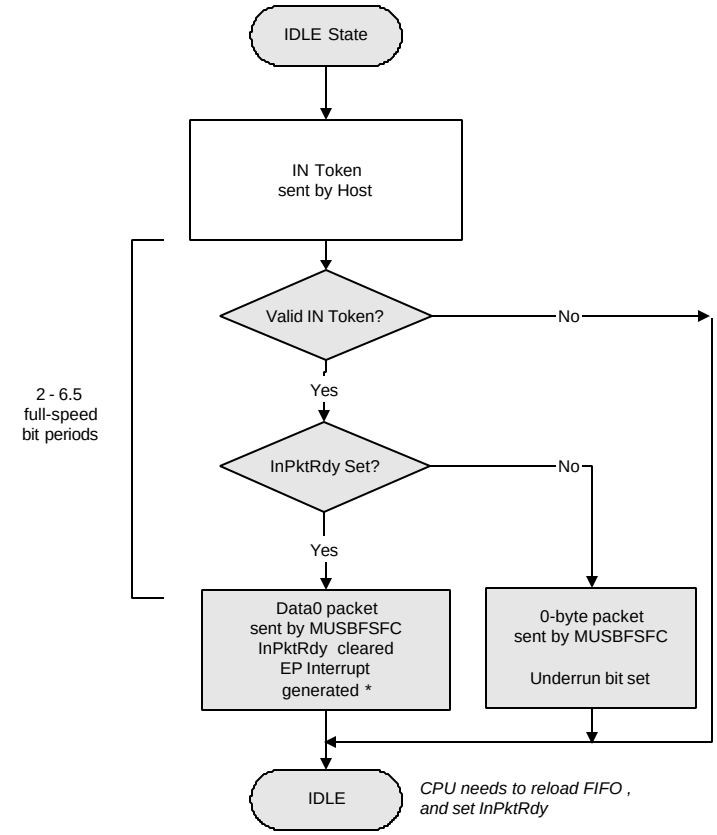


CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

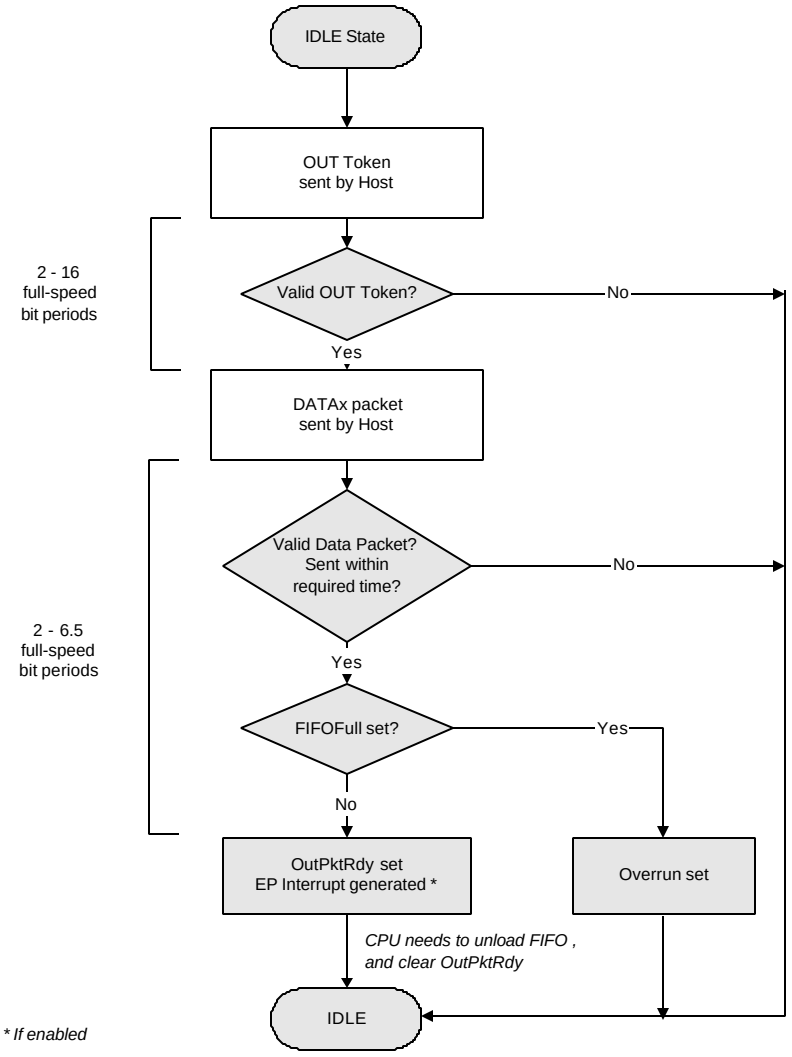
13.3. ISOCHRONOUS TRANSACTIONS

13.3.1. IN TRANSACTION



© 2000-2002 Mentor Graphics Corporation.

13.3.2. OUT TRANSACTION



CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

14. FIRMWARE

The firmware provided with the MUSBFSFC is an example of an interrupt handler for the MUSBFSFC interrupt. It shows how most of the *Universal Serial Bus Specification* Revision 2.0, Chapter 9 commands can be handled and also provides an example of how Bulk IN and OUT data endpoints could be handled.

Chapter 9 commands handled by the example firmware are:

SET_ADDRESS
GET_DESCRIPTOR: DEVICE, CONFIGURATION
SET_CONFIGURATION
SET_INTERFACE
GET_CONFIGURATION
GET_INTERFACE

SET_FEATURE commands and class or vendor specific commands can easily be added.

The firmware supports multiple configurations, the details of which are defined by constants in the **config.h** file. Macros for reading/writing the MUSBFSFC registers are defined in the **musbfsfc.h** file. Routines for loading/unloading the FIFOs are in a separate module, **fifo.c**, so they can easily be modified if (for example) DMA is to be used for accessing the FIFOs.

The following files are provided (listed on the following pages):

musbfsfc.c	Top level of the MUSBFSFC interrupt service routine
desc.c	Initialization of standard descriptors
fifo.c	MUSBFSFC FIFO read/write routines
endpoint0.c	Handler for Endpoint 0
endpoint.c	Handler for data endpoints
reset.c	USB reset routine
config.h	Header file containing constants and structures for the user's particular configuration
descript.h	Header file containing constants and structures for descriptor handling
endpoint0.h	Header file containing constants and structures for Endpoint 0
endpoint.h	Header file containing constants and structures for data endpoints
musbfsfc.h	Header file containing MUSBFSFC constants and register read/write macros

Note: The following files need to be modified by the user for their configuration:

config.h	Constants need to be modified for the users descriptors. Structures need to be defined for each configuration supported.
musbfsfc.h	The macros for MUSBFSFC register read/write and the register base address need to be defined.
fifo.c	The MUSBFSFC FIFO read/write routines may need to be modified.

You should also note that the descriptor structures defined in **config.h** assume 16-bit words are stored little-endian. If a big-endian processor is used, the byte order will need to be swapped.

© 2000-2002 Mentor Graphics Corporation.

14.1. ROUTINES

MUSBFSFC.C

```
/****************************************
  musbfsfc.c
  Interrupt handler for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/
#include "musbfsfc.h"
#include "descript.h"
#include "config.h"
#include "endpoint.h"

/* Device descriptor is global */
STD_DEV_DSCR        stddevdsc;

/* Configuration structures as defined in config.h */
M_CFG_1             cfg1;
M_CFG_2             cfg2;
M_CFG               cfg;

/* Current device state */
int                 gnDevState = DEVSTATE_DEFAULT;

/* Pointer to current configuration descriptor */
void *              gpCurCfg = NULL;

/* Pointer to current interface descriptors */
PSTD_IF_DSCR        gpCurIf[M_MAXIFS];

/* Current alternate interface values */
BYTE                gbyCurIfVal[M_MAXIFS];

/* Status for 2 Bulk In endpoints */
M_EPBIN_STATUS      gepbin1, gepbin2;

/* Status for 2 Bulk Out endpoints */
M_EPBOUT_STATUS     gepbout1, gepbout2;

/****************************************
  Top level interrupt handler
****************************************/
void musbfsfc_intr ()
{
BYTE                IntrUSB;
WORD                IntrIn;
WORD                IntrOut;

/* Read interrupt registers */
/* Mote if less than 8 IN endpoints are configured then */
/* only M_REG_INTRIN1 need be read. */
/* Similarly if less than 8 OUT endpoints are configured then */
/* only M_REG_INTROUT1 need be read. */
IntrUSB = MREAD_BYTE(M_REG_INTRUSB);
IntrIn  = (WORD)MREAD_BYTE(M_REG_INTRIN2);
IntrIn <<= 8;
IntrIn  |= (WORD)MREAD_BYTE(M_REG_INTRIN1);
IntrOut  = (WORD)MREAD_BYTE(M_REG_INTROUT2);
IntrOut <<= 8;
IntrOut  |= (WORD)MREAD_BYTE(M_REG_INTROUT1);

/* Check for resume from suspend mode */
/*
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
Add call to resume routine here
if (IntrUSB & M_INTR_RESUME) USBResume();
*/

/* Check for system interrupts */
if (IntrUSB & M_INTR_RESET) USBReset();

/* Check for endpoint 0 interrupt */
if (IntrIn & M_INTR_EP0) Endpoint0 (M_EP_NORMAL);

/* Check for each configured endpoint interrupt */
/* This example assumes 2 bulk IN endpoints with endpoint numbers 1,2, */
/* and 2 bulk out endpoints with endpoint numbers 1,2 */
if (IntrIn & 2) EndpointBulkIn (&gepbin1, M_EP_NORMAL);
if (IntrIn & 4) EndpointBulkIn (&gepbin2, M_EP_NORMAL);
if (IntrOut & 2) EndpointBulkOut (&gepbout1, M_EP_NORMAL);
if (IntrOut & 4) EndpointBulkOut (&gepbout2, M_EP_NORMAL);

/* Check for suspend mode */
/*
Add call to suspend routine here
if (IntrUSB & M_INTR_RESUME) USBSuspend();
*/

return;
}
```

DESC.C

```
/****************************************
  desc.c
  Initialise descriptors for MUSBFSFC firmware.

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/
#include "musbfsfc.h"
#include "endpoint0.h"
#include "descript.h"
#include "config.h"

extern STD_DEV_DSCR stddevdsc;

/* Add configuration structures as defined in config.h */
extern M_CFG_1      cfg1;
extern M_CFG_2      cfg2;
extern M_CFG        cfg;

/****************************************
  Initialise the descriptors
  This routine is called after a USB reset to set the descriptors.
****************************************/
void InitialiseDesc ()
{

/* Initialise the Standard Device Descriptor */
stddevdsc.bLength                = sizeof(STD_DEV_DSCR);
stddevdsc.bDescriptorType        = M_DST_DEVICE;
stddevdsc.bcdUSB                 = 0x0200;          /* USB spec rev is 2.0 */
stddevdsc.bMaxPacketSize0        = M_EP0_MAXP;
stddevdsc.idVendor               = M_DEVDSC_VENDOR;
stddevdsc.idProduct              = M_DEVDSC_PRODUCT;
stddevdsc.bcdDevice              = M_DEVDSC_RELEASE;
stddevdsc.iManufacturer          = M_DEVDSC_MANUI;
stddevdsc.iProduct               = M_DEVDSC_PRODI;
stddevdsc.iSerialNumber          = M_DEVDSC_SERIALI;
```


© 2000-2002 Mentor Graphics Corporation.

```
stddevdsc.bDeviceClass           = M_DEVDSC_CLASS;
stddevdsc.bDeviceSubClass        = M_DEVDSC_SUBCLASS;
stddevdsc.bDeviceProtocol        = M_DEVDSC_PROTOCOL;
stddevdsc.bNumConfigurations     = M_DEVDSC_CONFIGS;


/* Initialise the descriptors for configuration 1 */
cfg.pcfg1 = &cfg1;
cfg1.stdcfg.bLength              = sizeof(STD_CFG_DSCR);
cfg1.stdcfg.bDescriptorType      = M_DST_CONFIG;
cfg1.stdcfg.wTotalLength         = sizeof(M_CFG_1);
cfg1.stdcfg.bNumInterfaces       = M_CFG1_NUMIFS;
cfg1.stdcfg.bConfigurationValue  = 1;
cfg1.stdcfg.iConfiguration       = M_CFG1_CFGI;
cfg1.stdcfg.bmAttributes         = M_CFG1_ATTRIBUTES;
cfg1.stdcfg.bMaxPower            = M_CFG1_MAXPOWER;

/* .. Interface 0 */
cfg1.stdif00.bLength             = sizeof(STD_IF_DSCR);
cfg1.stdif00.bDescriptorType     = M_DST_INTERFACE;
cfg1.stdif00.bInterfaceNumber    = 0;
cfg1.stdif00.bAlternateSetting   = 0;
cfg1.stdif00.bNumEndpoints       = M_CFG1_I0A0_NUMEPS;
cfg1.stdif00.bInterfaceClass     = M_CFG1_I0A0_IFCLASS;
cfg1.stdif00.bInterfaceSubClass  = M_CFG1_I0A0_IFSUBCLASS;
cfg1.stdif00.bInterfaceProtocol  = M_CFG1_I0A0_IFPROTOCOL;
cfg1.stdif00.iInterface          = M_CFG1_I0A0_IFI;

/* .. Endpoints */
cfg1.stdep00_0.bLength           = sizeof(STD_EP_DSCR);
cfg1.stdep00_0.bDescriptorType   = M_DST_ENDPOINT;
cfg1.stdep00_0.bEndpointAddress  = M_CFG1_I0A0_0_EPADDR;
cfg1.stdep00_0.bmAttributes      = M_CFG1_I0A0_0_ATTRIB;
cfg1.stdep00_0.wMaxPacketSize    = M_CFG1_I0A0_0_MAXP;
cfg1.stdep00_0.bInterval         = M_CFG1_I0A0_0_INTER;
cfg1.stdep00_1.bLength           = sizeof(STD_EP_DSCR);
cfg1.stdep00_1.bDescriptorType   = M_DST_ENDPOINT;
cfg1.stdep00_1.bEndpointAddress  = M_CFG1_I0A0_1_EPADDR;
cfg1.stdep00_1.bmAttributes      = M_CFG1_I0A0_1_ATTRIB;
cfg1.stdep00_1.wMaxPacketSize    = M_CFG1_I0A0_1_MAXP;
cfg1.stdep00_1.bInterval         = M_CFG1_I0A0_1_INTER;

/* .. Interface 1, Alternate 0 */
cfg1.stdif10.bLength             = sizeof(STD_IF_DSCR);
cfg1.stdif10.bDescriptorType     = M_DST_INTERFACE;
cfg1.stdif10.bInterfaceNumber    = 1;
cfg1.stdif10.bAlternateSetting   = 0;
cfg1.stdif10.bNumEndpoints       = M_CFG1_I1A0_NUMEPS;
cfg1.stdif10.bInterfaceClass     = M_CFG1_I1A0_IFCLASS;
cfg1.stdif10.bInterfaceSubClass  = M_CFG1_I1A0_IFSUBCLASS;
cfg1.stdif10.bInterfaceProtocol  = M_CFG1_I1A0_IFPROTOCOL;
cfg1.stdif10.iInterface          = M_CFG1_I1A0_IFI;

/* .. Endpoints */
cfg1.stdep10_0.bLength           = sizeof(STD_EP_DSCR);
cfg1.stdep10_0.bDescriptorType   = M_DST_ENDPOINT;
cfg1.stdep10_0.bEndpointAddress  = M_CFG1_I1A0_0_EPADDR;
cfg1.stdep10_0.bmAttributes      = M_CFG1_I1A0_0_ATTRIB;
cfg1.stdep10_0.wMaxPacketSize    = M_CFG1_I1A0_0_MAXP;
cfg1.stdep10_0.bInterval         = M_CFG1_I1A0_0_INTER;
cfg1.stdep10_1.bLength           = sizeof(STD_EP_DSCR);
cfg1.stdep10_1.bDescriptorType   = M_DST_ENDPOINT;
cfg1.stdep10_1.bEndpointAddress  = M_CFG1_I1A0_1_EPADDR;
cfg1.stdep10_1.bmAttributes      = M_CFG1_I1A0_1_ATTRIB;
cfg1.stdep10_1.wMaxPacketSize    = M_CFG1_I1A0_1_MAXP;
cfg1.stdep10_1.bInterval         = M_CFG1_I1A0_1_INTER;
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
/* .. Interface 1, Alternate 1 */
cfg1.stdif11.bLength              = sizeof(STD_IF_DSCR);
cfg1.stdif11.bDescriptorType      = M_DST_INTERFACE;
cfg1.stdif11.bInterfaceNumber     = 1;
cfg1.stdif11.bAlternateSetting    = 1;
cfg1.stdif11.bNumEndpoints        = M_CFG1_I1A1_NUMEPS;
cfg1.stdif11.bInterfaceClass      = M_CFG1_I1A1_IFCLASS;
cfg1.stdif11.bInterfaceSubClass   = M_CFG1_I1A1_IFSUBCLASS;
cfg1.stdif11.bInterfaceProtocol   = M_CFG1_I1A1_IFPROTOCOL;
cfg1.stdif11.iInterface           = M_CFG1_I1A1_IFI;

/* .. Endpoints */
cfg1.stdep11_0.bLength            = sizeof(STD_EP_DSCR);
cfg1.stdep11_0.bDescriptorType    = M_DST_ENDPOINT;
cfg1.stdep11_0.bEndpointAddress   = M_CFG1_I1A1_0_EPADDR;
cfg1.stdep11_0.bmAttributes       = M_CFG1_I1A1_0_ATTRIB;
cfg1.stdep11_0.wMaxPacketSize     = M_CFG1_I1A1_0_MAXP;
cfg1.stdep11_0.bInterval          = M_CFG1_I1A1_0_INTER;
cfg1.stdep11_1.bLength            = sizeof(STD_EP_DSCR);
cfg1.stdep11_1.bDescriptorType    = M_DST_ENDPOINT;
cfg1.stdep11_1.bEndpointAddress   = M_CFG1_I1A1_1_EPADDR;
cfg1.stdep11_1.bmAttributes       = M_CFG1_I1A1_1_ATTRIB;
cfg1.stdep11_1.wMaxPacketSize     = M_CFG1_I1A1_1_MAXP;
cfg1.stdep11_1.bInterval          = M_CFG1_I1A1_1_INTER;


/* Initialise the descriptors for configuration 2 */
cfg.pcfg2 = &cfg2;
cfg2.stdcfg.bLength               = sizeof(STD_CFG_DSCR);
cfg2.stdcfg.bDescriptorType       = M_DST_CONFIG;
cfg2.stdcfg.wTotalLength          = sizeof(M_CFG_2);
cfg2.stdcfg.bNumInterfaces        = M_CFG2_NUMIFS;
cfg2.stdcfg.bConfigurationValue   = 2;
cfg2.stdcfg.iConfiguration        = M_CFG2_CFGI;
cfg2.stdcfg.bmAttributes          = M_CFG2_ATTRIBUTES;
cfg2.stdcfg.bMaxPower             = M_CFG2_MAXPOWER;

/* .. Interface 0 */
cfg2.stdif00.bLength              = sizeof(STD_IF_DSCR);
cfg2.stdif00.bDescriptorType      = M_DST_INTERFACE;
cfg2.stdif00.bInterfaceNumber     = 0;
cfg2.stdif00.bAlternateSetting    = 0;
cfg2.stdif00.bNumEndpoints        = M_CFG2_I0A0_NUMEPS;
cfg2.stdif00.bInterfaceClass      = M_CFG2_I0A0_IFCLASS;
cfg2.stdif00.bInterfaceSubClass   = M_CFG2_I0A0_IFSUBCLASS;
cfg2.stdif00.bInterfaceProtocol   = M_CFG2_I0A0_IFPROTOCOL;
cfg2.stdif00.iInterface           = M_CFG2_I0A0_IFI;

/* .. Endpoints */
cfg2.stdep00_0.bLength            = sizeof(STD_EP_DSCR);
cfg2.stdep00_0.bDescriptorType    = M_DST_ENDPOINT;
cfg2.stdep00_0.bEndpointAddress   = M_CFG2_I0A0_0_EPADDR;
cfg2.stdep00_0.bmAttributes       = M_CFG2_I0A0_0_ATTRIB;
cfg2.stdep00_0.wMaxPacketSize     = M_CFG2_I0A0_0_MAXP;
cfg2.stdep00_0.bInterval          = M_CFG2_I0A0_0_INTER;
cfg2.stdep00_1.bLength            = sizeof(STD_EP_DSCR);
cfg2.stdep00_1.bDescriptorType    = M_DST_ENDPOINT;
cfg2.stdep00_1.bEndpointAddress   = M_CFG2_I0A0_1_EPADDR;
cfg2.stdep00_1.bmAttributes       = M_CFG2_I0A0_1_ATTRIB;
cfg2.stdep00_1.wMaxPacketSize     = M_CFG2_I0A0_1_MAXP;
cfg2.stdep00_1.bInterval          = M_CFG2_I0A0_1_INTER;

/* .. Interface 1, Alternate 0 */
cfg2.stdif10.bLength              = sizeof(STD_IF_DSCR);
cfg2.stdif10.bDescriptorType      = M_DST_INTERFACE;
cfg2.stdif10.bInterfaceNumber     = 1;
cfg2.stdif10.bAlternateSetting    = 0;
```


© 2000-2002 Mentor Graphics Corporation.

```
cfg2.stdif10.bNumEndpoints       = M_CFG2_I1A0_NUMEPS;
cfg2.stdif10.bInterfaceClass     = M_CFG2_I1A0_IFCLASS;
cfg2.stdif10.bInterfaceSubClass  = M_CFG2_I1A0_IFSUBCLASS;
cfg2.stdif10.bInterfaceProtocol  = M_CFG2_I1A0_IFPROTOCOL;
cfg2.stdif10.iInterface          = M_CFG2_I1A0_IFI;

/* .. Endpoints */
cfg2.stdep10_0.bLength           = sizeof(STD_EP_DSCR);
cfg2.stdep10_0.bDescriptorType   = M_DST_ENDPOINT;
cfg2.stdep10_0.bEndpointAddress  = M_CFG2_I1A0_0_EPADDR;
cfg2.stdep10_0.bmAttributes      = M_CFG2_I1A0_0_ATTRIB;
cfg2.stdep10_0.wMaxPacketSize    = M_CFG2_I1A0_0_MAXP;
cfg2.stdep10_0.bInterval         = M_CFG2_I1A0_0_INTER;
cfg2.stdep10_1.bLength           = sizeof(STD_EP_DSCR);
cfg2.stdep10_1.bDescriptorType   = M_DST_ENDPOINT;
cfg2.stdep10_1.bEndpointAddress  = M_CFG2_I1A0_1_EPADDR;
cfg2.stdep10_1.bmAttributes      = M_CFG2_I1A0_1_ATTRIB;
cfg2.stdep10_1.wMaxPacketSize    = M_CFG2_I1A0_1_MAXP;
cfg2.stdep10_1.bInterval         = M_CFG2_I1A0_1_INTER;

}
```

ENDPOINT0.C

```
/****************************************
  endpoint0.c
  Endpoint 0 interrupt routine for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/
#include "musbfsfc.h"
#include "endpoint0.h"
#include "descript.h"
#include "config.h"

extern STD_DEV_DSCR stddevdsc;

/* Add configuration structures as defined in config.h */
extern M_CFG_1      cfg1;
extern M_CFG_2      fg2;
extern M_CFG        cfg;

/* Global status */
extern int          gnDevState;
extern void *       gpCurCfg;
extern PSTD_IF_DSCR gpCurIf[M_MAXIFS];
extern BYTE         gbyCurIfVal[M_MAXIFS];

/* Local functions */
void Endpoint0_Command (PM_EP0_STATUS, PCOMMAND);
void Endpoint0_Tx (PM_EP0_STATUS);
void StdDevReq (PM_EP0_STATUS, PCOMMAND);
int ConfigureIfs ();


/****************************************
  Endpoint 0 interrupt service routine
****************************************/
void Endpoint0 (int nCallState)
{
static M_EP0_STATUS ep0state;
static COMMAND      cmd;
BYTE                byCSR0;
```


© 2000-2002 Mentor Graphics Corporation.

```
/* Check for USB reset of endpoint 0 */
if (nCallState == M_EP_RESET) {
  ep0state.nState = M_EP0_IDLE;
  ep0state.byFAddr = 0xFF;
  /* Clear current configuration pointer */
  gpCurCfg = NULL;
  return;
  }

/* Check for status stage of a request */
if (!(byCSR0 & M_CSR0_OUTPKTRDY)) {
  /* Complete SET_ADDRESS command */
  if (ep0state.byFAddr != 0xFF) {
    MWRITE_BYTE(M_REG_FADDR, ep0state.byFAddr);
    if ((gnDevState == DEVSTATE_DEFAULT) && ep0state.byFAddr)
      gnDevState = DEVSTATE_ADDRESS;
    else if ((gnDevState == DEVSTATE_ADDRESS) && !ep0state.byFAddr)
      gnDevState = DEVSTATE_DEFAULT;
    }
  }
/* Clear pending commands */
ep0state.byFAddr = 0xFF;

/* Read CSR0 */
MWRITE_BYTE(M_REG_INDEX, 0);
byCSR0 = MREAD_BYTE(M_REG_CSR0);

/* Check for SentStall */
if (byCSR0 & M_CSR0_SENTSTALL) {
  MWRITE_BYTE(M_REG_CSR0, (byCSR0 & ~M_CSR0_SENDSTALL));
  ep0state.nState = M_EP0_IDLE;
  }

/* Check for SetupEnd */
if (byCSR0 & M_CSR0_SETUPEND) {
  MWRITE_BYTE(M_REG_CSR0, (byCSR0 & M_CSR0_SVDSETUPEND));
  ep0state.nState = M_EP0_IDLE;
  }

/* Call relevant routines for endpoint 0 state */

if (ep0state.nState == M_EP0_IDLE) {
  /* If no packet has been received, */
  /* assume that this was a STATUS phase complete. */
  /* Otherwise load new comand */
  if (byCSR0 & M_CSR0_OUTPKTRDY) {
    /* Read the 8-byte command from the FIFO */
    /* There is no need to check that OutCount is set to 8 */
    /* as the MUSBFSFC will reject SETUP packets that are not 8 bytes long. */
    FIFORead (0, 8, &cmd);
    Endpoint0_Command (&ep0state, &cmd);
    }
  }
if (ep0state.nState == M_EP0_TX)
  Endpoint0_Tx(&ep0state);

return;
}
```


© 2000-2002 Mentor Graphics Corporation.

```
/****************************************
  Endpoint 0 command
  This routine is called when a command has been received in the SETUP phase.
****************************************/
void Endpoint0_Command (PM_EP0_STATUS pep0state, PCOMMAND pcmd)
{
BYTE                byType;

/* Check request type */
switch (byType = pcmd->bmRequestType & M_CMD_TYPEMASK) {

  case M_CMD_STDREQ:
    StdDevReq (pep0state, pcmd);
    break;
  case M_CMD_CLASSREQ:
    /* Add call to external routine for handling class requests */
  case M_CMD_VENDREQ:
    /* Add call to external routine for handling vendor requests */
  default:
    /* Stall the command if a reserved request is received */
    MWRITE_BYTE(M_REG_CSR0, M_CSR0_SVDOUTPKTRDY | M_CSR0_SENDSTALL);
    break;
  }

return;
}

/****************************************
  Standard Device Request
  This routine is called when a standard device request has been received.
  The bRequest field of the received command is decoded.
****************************************/
void StdDevReq (PM_EP0_STATUS pep0state, PCOMMAND pcmd)
{
int                 bError = FALSE, n;
BYTE                by;
BYTE                byConfig;
PSTD_CFG_DSCR       pcfg;
BYTE *              *ppby;

switch (pcmd->bRequest) {

  case SET_ADDRESS:
    /* Store device function address until status stage of request */
    if (pcmd->bmRequestType != M_CMD_STDDEVOUT)
      bError = TRUE;
    else if (gnDevState <= DEVSTATE_ADDRESS)
      pep0state->byFAddr = (BYTE)pcmd->wValue;
    else
      bError = TRUE;
    break;

  case GET_DESCRIPTOR:
    /* Decode the required descriptor from the command */
    if (pcmd->bmRequestType != M_CMD_STDDEVIN)
      bError = TRUE;
    else {
      switch (pcmd->wValue & M_CMD_DESCMASK) {
        case M_CMD_DEVICE:
          /* Prepare to return Standard Device Descriptor */
          pep0state->nBytesLeft = sizeof(STD_DEV_DSCR);
          /* Check host is allowing a descriptor this long */
          if (pcmd->wLength < pep0state->nBytesLeft)
            pep0state->nBytesLeft = pcmd->wLength;
          pep0state->pData = (void*)&stddevdsc;
          pep0state->nState = M_EP0_TX;
          break;
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
        case M_CMD_DEVQUAL:
          /* No alternate speed supported */
          bError = TRUE;
          break;
        case M_CMD_CONFIG:
          byConfig = (BYTE)(pcmd->wValue & 0x00FF);
          if (byConfig >= stddevdsc.bNumConfigurations)
            bError = TRUE;
          else {
            /* Get pointer to requested configuration descriptor */
            ppby = (BYTE**)&cfg;
            ppby += byConfig;
            pcfg = (PSTD_CFG_DSCR)*ppby;
            /* Prepare to return Configuration Descriptors */
            pep0state->nBytesLeft = (int)pcfg->wTotalLength;
            pep0state->pData = (void*)pcfg;
            pep0state->nState = M_EP0_TX;
            /* Check host is allowing a descriptor this long */
            if (pcmd->wLength < pep0state->nBytesLeft)
              pep0state->nBytesLeft = pcmd->wLength;
            }
          break;
        case M_CMD_OTHERSPEED:
          /* No alternate speed supported */
          bError = TRUE;
          break;
        default: bError = TRUE;
        }
      }
    break;
  case SET_CONFIGURATION:
    byConfig = (BYTE)(pcmd->wValue & 0x00FF);
    if (gnDevState == DEVSTATE_DEFAULT)
      bError = TRUE;
    /* Assumes configurations are numbered 1 to NumConfigurations */
    else if (byConfig > stddevdsc.bNumConfigurations)
      bError = TRUE;
    else if (!byConfig)
      gnDevState = DEVSTATE_ADDRESS;
    else {
      /* Get pointer to requested configuration descriptor */
      ppby = (BYTE**)&cfg;
      ppby += byConfig-1;
      gpCurCfg = (void *)*ppby;
      /* Set all alternate settings to zero */
      for (n=0; n<M_MAXIFS; n++)
        gbyCurIfVal[n] = 0;
      /* Configure endpoints */
      ConfigureIfs ();
      gnDevState = DEVSTATE_CONFIG;
      MWRITE_BYTE(M_REG_INDEX, 0);
      }
    break;
  case SET_INTERFACE:
    if (gnDevState > DEVSTATE_ADDRESS) {
      n = (int)pcmd->wIndex;
      pcfg = (PSTD_CFG_DSCR)gpCurCfg;
      if ((BYTE)n >= pcfg->bNumInterfaces)
        bError = TRUE;
      else {
        gbyCurIfVal[n] = (BYTE)pcmd->wValue;
        if (ConfigureIfs() == FALSE)
          bError = TRUE;
        MWRITE_BYTE(M_REG_INDEX, 0);
        }
      }
```


© 2000-2002 Mentor Graphics Corporation.

```
    else
      bError = TRUE;
    break;
  case GET_CONFIGURATION:
    if ((gnDevState == DEVSTATE_ADDRESS) && (!pcmd->wValue)) {
      /* Prepare to return zero */
      pep0state->nBytesLeft = 1;
      pep0state->pData = (void*)&pcmd->wValue;
      pep0state->nState = M_EP0_TX;
      }
    else if (gnDevState > DEVSTATE_ADDRESS) {
      /* Prepare to return configuration value */
      pcfg = (PSTD_CFG_DSCR)gpCurCfg;
      pep0state->nBytesLeft = 1;
      pep0state->pData = (void*)&pcfg->bConfigurationValue;
      pep0state->nState = M_EP0_TX;
      }
    else
      bError = TRUE;
    break;
  case GET_INTERFACE:
    if (gnDevState > DEVSTATE_ADDRESS) {
      n = (int)pcmd->wIndex;
      pcfg = (PSTD_CFG_DSCR)gpCurCfg;
      if ((BYTE)n >= pcfg->bNumInterfaces)
        bError = TRUE;
      else {
        /* Prepare to return interface value */
        pep0state->nBytesLeft = 1;
        pep0state->pData = (void*)&gpCurIf[n]->bInterfaceNumber;
        pep0state->nState = M_EP0_TX;
        }
      }
    else
      bError = TRUE;
    break;
  case SET_FEATURE:
    switch (pcmd->bmRequestType) {
      case M_CMD_STDDEVOUT:
        switch (pcmd->wValue) {
          case M_FTR_DEVREMWAKE:
          default:
            bError = TRUE;
            break;
          }
        break;
      case M_CMD_STDIFIN:
        /* Add interface features here */
      case M_CMD_STDEPIN:
        /* Add endpoint features here */
      default:
        bError = TRUE;
        break;
      }
    break;
  default:
    /* Stall the command if an unrecognized request is received */
    bError = TRUE;
    break;
  }
/* Stall command if an error occured */
if (bError)
  by = M_CSR0_SVDOUTPKTRDY | M_CSR0_SENDSTALL;
else
  by = M_CSR0_SVDOUTPKTRDY;
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
MWRITE_BYTE(M_REG_CSR0, by);

return;
}
/****************************************
  Endpoint 0 Tx
  This routine is called when data is to be sent to the host.
****************************************/
void Endpoint0_Tx (PM_EP0_STATUS pep0state)
{
int                 nBytes;
BYTE                by;

/* Determine number of bytes to send */
if (pep0state->nBytesLeft <= M_EP0_MAXP) {
  nBytes = pep0state->nBytesLeft;
  pep0state->nBytesLeft = 0;
  }
else {
  nBytes = M_EP0_MAXP;
  pep0state->nBytesLeft -= M_EP0_MAXP;
  }

FIFOWrite (0, nBytes, pep0state->pData);
pep0state->pData = (BYTE *)pep0state->pData + nBytes;
if (nBytes < M_EP0_MAXP) {
  by = M_CSR0_INPKTRDY | M_CSR0_DATAEND;
  pep0state->nState = M_EP0_IDLE;
  }
else
  by = M_CSR0_INPKTRDY;
MWRITE_BYTE(M_REG_CSR0, by);

return;
}

/****************************************
  ConfigureIfs
  This routine is called when a SET_CONFIGURATION or SET_INTERFACE
  command is received and will set the gpCurIF pointers
  to the selected interface descriptors and will set the
  maximum packet size and operating mode for the endpoints in
  the selected interfaces
****************************************/
int ConfigureIfs ()
{
PSTD_CFG_DSCR       pcfg;
BYTE                byIf, byAltIf, byEP, byNumEPs, by;
BYTE                *pby;
BYTE                *pbyIfVal;
PSTD_IF_DSCR        pif;
PSTD_EP_DSCR        pep;

/* Set pointer to first interface descriptor in current configuration */
pby = (BYTE *)gpCurCfg;
pby += sizeof(STD_CFG_DSCR);
pif = (PSTD_IF_DSCR)pby;

/* Loop through all interfaces in the current configuration */
pcfg = (PSTD_CFG_DSCR)gpCurCfg;
pbyIfVal = (BYTE*)&gbyCurIfVal;
for (byIf=0; byIf<pcfg->bNumInterfaces; byIf++, pbyIfVal++) {

  /* Advance pointer to selected alternate interface descriptor */
  if (*pbyIfVal) {
    for (byAltIf=0; byAltIf<*pbyIfVal; byAltIf++) {
      byNumEPs = pif->bNumEndpoints;
      pby += sizeof(STD_IF_DSCR) + byNumEPs * sizeof(STD_EP_DSCR);
      pif = (PSTD_IF_DSCR)pby;
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
      /* Check an alternate setting > number of alternates not specified */
      if (!pif->bAlternateSetting)
        return FALSE;
      }
    }
/* Store pointer to interface in global array */
  gpCurIf[byIf] = pif;
  /* Loop through all endpoints in interface */
  byNumEPs = pif->bNumEndpoints;
  pby += sizeof(STD_IF_DSCR);
  for (byEP=0; byEP<byNumEPs; byEP++) {
    pep = (PSTD_EP_DSCR)pby;
    /* Configure the endpoint */
    MWRITE_BYTE(M_REG_INDEX, (pep->bEndpointAddress & 0x0F));
    if (pep->bEndpointAddress & 0x80) {
      if (pep->wMaxPacketSize == 1023)
        by = 128;
      else
        by = (BYTE)(pep->wMaxPacketSize >> 3);
      MWRITE_BYTE(M_REG_INMAXP, by);
      by = MREAD_BYTE(M_REG_INCSR2);
      switch (pep->bmAttributes & M_EP_TFMASK) {
        case M_EP_ISO:
          by |= M_INCSR2_ISO;
          break;
        case M_EP_BULK:
        case M_EP_INTR:
          by &= ~M_INCSR2_ISO;
          break;
        }
      /* Set mode bit high (only strictly necessary if sharing a FIFO) */
      by |= M_INCSR2_MODE;
      MWRITE_BYTE(M_REG_INCSR2, by);
      /* Other configuration for an IN endpoint */
      /* e.g. AutoClr, DMA */
      /* should be added here */
      }
    else {
      if (pep->wMaxPacketSize == 1023)
        by = 128;
      else
        by = (BYTE)(pep->wMaxPacketSize >> 3);
      MWRITE_BYTE(M_REG_OUTMAXP, by);
      by = MREAD_BYTE(M_REG_OUTCSR2);
      switch (pep->bmAttributes & M_EP_TFMASK) {
        case M_EP_ISO:
          by |= M_OUTCSR2_ISO;
          break;
        case M_EP_BULK:
        case M_EP_INTR:
          by &= ~M_OUTCSR2_ISO;
          break;
        }
      MWRITE_BYTE(M_REG_OUTCSR2, by);
      /* Other configuration for an OUT endpoint */
      /* e.g. AutoSet, DMA */
      /* should be added here */
      }
    pby += sizeof(STD_EP_DSCR);
    }
  /* Skip any alternate setting interfaces */
  pif = (PSTD_IF_DSCR)pby;
  while (pif->bAlternateSetting) {
    byNumEPs = pif->bNumEndpoints;
    pby += sizeof(STD_IF_DSCR) + byNumEPs * sizeof(STD_EP_DSCR);
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
    pif = (PSTD_IF_DSCR)pby;
    }
  }
return TRUE;
}
```

ENDPOINT.C

```
/****************************************
  endpoint.c
  Endpoint interrupt routine for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/
#include "musbfsfc.h"
#include "endpoint.h"

/* Global status */
extern int          gnDevState;

/****************************************
  EndpointBulkIn service routine
  This routine provides simple handling for bulk in endpoints.
  Endpoint MaxP is defined in endpoint.h.
  While the nBytesLeft field of the pbostate structure is >= 0
  the endpoint will send packets of MaxP, with the residue in the last packet.
  If the data to send is an exact multiple of MaxP then a null data packet
  will be sent.
****************************************/
void EndpointBulkIn (PM_EPBIN_STATUS pbistate, int nCallState)
{
int      nBytes;
BYTE     byInCSR;

/* Reset endpoint */
if (nCallState == M_EP_RESET) {
  pbistate->nBytesLeft = M_EP_NODATA;
  MWRITE_BYTE(M_REG_INDEX, pbistate->byEP);
  byInCSR = M_INCSR_CDT | M_INCSR_FF;
  MWRITE_BYTE(M_REG_INCSR1, byInCSR);
  return;
  }

/* Check whether there is any data to send */
if (pbistate->nBytesLeft == M_EP_NODATA)
  return;

/* Determine number of bytes to send */
if (pbistate->nBytesLeft < M_EP_MAXP) {
  nBytes = pbistate->nBytesLeft;
  pbistate->nBytesLeft = M_EP_NODATA;
  }
else {
  nBytes = M_EP_MAXP;
  pbistate->nBytesLeft -= M_EP_MAXP;
  }

/* Load FIFO */
FIFOWrite ((int)pbistate->byEP, nBytes, pbistate->pData);
pbistate->pData = (BYTE *)pbistate->pData + nBytes;

MWRITE_BYTE(M_REG_INDEX, pbistate->byEP);
byInCSR = M_INCSR_IPR;
MWRITE_BYTE(M_REG_INCSR1, byInCSR);

return;
}
```


© 2000-2002 Mentor Graphics Corporation.

```
/****************************************
  EndpointBulkOut service routine
  This routine provides simple handling for bulk out endpoints.
  Endpoint MaxP is defined in endpoint.h.
  It assumes a transfer is complete when a packet of less than MaxP is received.
****************************************/
void EndpointBulkOut (PM_EPBOUT_STATUS pbostate, int nCallState)
{
int      nBytes;
BYTE     byOutCSR;

/* Reset endpoint */
if (nCallState == M_EP_RESET) {
  pbostate->nBytesRecv = 0;
  pbostate->nBuffLen = 0;
  MWRITE_BYTE(M_REG_INDEX, pbostate->byEP);
  byOutCSR = M_OUTCSR_CDT | M_OUTCSR_FF;
  MWRITE_BYTE(M_REG_OUTCSR1, byOutCSR);
  return;
  }

/* Set index register */
MWRITE_BYTE(M_REG_INDEX, pbostate->byEP);

/* Read OUT CSR register */
byOutCSR = MREAD_BYTE(M_REG_OUTCSR1);

/* Get packet, */
/* may need to unload more than one packet if double buffering enabled */
while (byOutCSR & M_OUTCSR_OPR) {

  /* Get packet size */
  nBytes = (int)MREAD_BYTE(M_REG_OUTCOUNT2);
  nBytes <<= 8;
  nBytes |= (int)MREAD_BYTE(M_REG_OUTCOUNT1);

  /* Check there is room in the buffer */
  if (pbostate->nBytesRecv + nBytes > pbostate->nBuffLen) {
    /* Call to function to handle buffer over run */
    MWRITE_BYTE(M_REG_OUTCSR1, 0);
    return;
    }

  /* Unload FIFO */
  FIFORead ((int)pbostate->byEP, nBytes, pbostate->pData);

  /* Update status */
  pbostate->nBytesRecv += nBytes;
  pbostate->pData = (BYTE *)pbostate->pData + nBytes;

  /* Check for end of transfer */
  if (nBytes < M_EP_MAXP) {
    /* Call function to handle received data */
    }

  /* Clear OutPktRdy */
  MWRITE_BYTE(M_REG_OUTCSR1, 0);

  /* Check for another packet */
  byOutCSR = MREAD_BYTE(M_REG_OUTCSR1);
  }

return;
}
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

FIFO.C

```
/****************************************

  fifo.c
  FIFO access routines for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000

****************************************/
#include "musbfsfc.h"


/****************************************
  FIFORead
****************************************/
void FIFORead (int nEP, int nBytes, void * pDst)
{
int                 nCount;
BYTE *              pby;
int                 nAddr;

if (nBytes) {
  nAddr = M_FIFO_EP0+nEP;
  nCount = nBytes;
  pby = (BYTE *)pDst;
  while (nCount) {
    *pby++ = *((BYTE *)nAddr);
    nCount--;
    }
  }

return;
}


/****************************************
  FIFOWrite
****************************************/
void FIFOWrite (int nEP, int nBytes, void * pSrc)
{
int                 nCount;
BYTE *              pby;
int                 nAddr;

if (nBytes) {
  nAddr = M_FIFO_EP0+nEP;
  nCount = nBytes;
  pby = (BYTE *)pSrc;
  while (nCount) {
   *((BYTE *)nAddr) = *pby++;
    nCount--;
    }
  }

return;
}
```


© 2000-2002 Mentor Graphics Corporation.

RESET.C

```
/****************************************

  reset.c
  Reset interrupt routine for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000

****************************************/
#include "musbfsfc.h"
#include "endpoint.h"

extern int          gnDevState;

extern M_EPBIN_STATUS            gepbin1, gepbin2;
extern M_EPBOUT_STATUS           gepbout1, gepbout2;


/****************************************
  USB Reset
****************************************/
void USBReset ()
{

/* Set device into default state */
gnDevState = DEVSTATE_DEFAULT;

/* Initialise descriptors */
InitialiseDesc ();

/* Reset any state machines for each endpoint */
/* This example assumes 2 bulk IN endpoints with endpoint numbers 1,2, */
/* and 2 bulk out endpoints with endpoint numbers 1,2 */
Endpoint0 (M_EP_RESET);

/* Initialise endpoint status structures with endpoint numbers */
gepbin1.byEP = 1;
gepbin2.byEP = 2;
gepbout1.byEP = 1;
gepbout2.byEP = 2;

EndpointBulkIn (&gepbin1, M_EP_RESET);
EndpointBulkIn (&gepbin2, M_EP_RESET);
EndpointBulkOut (&gepbout1, M_EP_RESET);
EndpointBulkOut (&gepbout2, M_EP_RESET);

return;
}
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

14.2. HEADER FILES

CONFIG.H

```
/****************************************
  config.h
  Configuration for MUSBFSFC firmware
  This file defines the number of descriptors for the device.
  It must be modified by the user to match the required configuration.

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/

#ifndef _CONFIG_H_
#define _CONFIG_H_

/* Define maximum packet size for endpoint 0 */
#define M_EP0_MAXP  8

/* Define maximum number of interfaces in a configuration */
#define M_MAXIFS    16

/* Define values for Standard Device Descriptor */
#define M_DEVDSC_CLASS          0x00            /* Device Class */
#define M_DEVDSC_SUBCLASS       0x00            /* Device SubClass */
#define M_DEVDSC_PROTOCOL       0x00            /* Device Protocol */
#define M_DEVDSC_VENDOR         0x0000          /* Vendor ID */
#define M_DEVDSC_PRODUCT        0x0000          /* Product ID */
#define M_DEVDSC_RELEASE        0x0000          /* Release number */
#define M_DEVDSC_MANUI          0               /* Manufacturer string index */
#define M_DEVDSC_PRODI          0               /* Product string index */
#define M_DEVDSC_SERIALI        0               /* Serial number string index */
#define M_DEVDSC_CONFIGS        2               /* Number of configurations */


/* Define values for descriptors in configuration 1 */
#define M_CFG1_NUMIFS           2               /* Number of interfaces */
#define M_CFG1_CFGI             0               /* Configuration description string index
*/
#define M_CFG1_ATTRIBUTES       0x00            /* Configuration attributes */
#define M_CFG1_MAXPOWER         0               /* Maximum power (units of 2 mA) */

/* Interface 0, alternate 0 */
#define M_CFG1_I0A0_NUMEPS      2               /* Number of endpoints */
#define M_CFG1_I0A0_IFCLASS     0               /* Interface class */
#define M_CFG1_I0A0_IFSUBCLASS  0               /* Interface subclass */
#define M_CFG1_I0A0_IFPROTOCOL  0               /* Interface protocol */
#define M_CFG1_I0A0_IFI         0               /* Interface string index */
#define M_CFG1_I0A0_0_EPADDR    0x81            /* Endpoint address (endpoint 1 IN) */
#define M_CFG1_I0A0_0_ATTRIB    0x02            /* Endpoint attributes (bulk) */
#define M_CFG1_I0A0_0_MAXP      0x0040          /* Endpoint max packet size (64) */
#define M_CFG1_I0A0_0_INTER     0               /* Endpoint interval */
#define M_CFG1_I0A0_1_EPADDR    0x01            /* Endpoint address (endpoint 1 OUT) */
#define M_CFG1_I0A0_1_ATTRIB    0x02            /* Endpoint attributes (bulk) */
#define M_CFG1_I0A0_1_MAXP      0x0040          /* Endpoint max packet size (64) */
#define M_CFG1_I0A0_1_INTER     0               /* Endpoint interval */

/* Interface 1, alternate 0 */
#define M_CFG1_I1A0_NUMEPS      2               /* Number of endpoints */
#define M_CFG1_I1A0_IFCLASS     0               /* Interface class */
#define M_CFG1_I1A0_IFSUBCLASS  0               /* Interface subclass */
#define M_CFG1_I1A0_IFPROTOCOL  0               /* Interface protocol */
#define M_CFG1_I1A0_IFI         0               /* Interface string index */
```


© 2000-2002 Mentor Graphics Corporation.

```
#define M_CFG1_I1A0_0_EPADDR     0x82              /* Endpoint address (endpoint 2 IN) */
#define M_CFG1_I1A0_0_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG1_I1A0_0_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG1_I1A0_0_INTER      0                 /* Endpoint interval */
#define M_CFG1_I1A0_1_EPADDR     0x02              /* Endpoint address (endpoint 2 OUT) */
#define M_CFG1_I1A0_1_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG1_I1A0_1_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG1_I1A0_1_INTER      0                 /* Endpoint interval */

/* Interface 1, alternate 1 */
#define M_CFG1_I1A1_NUMEPS       2                 /* Number of endpoints */
#define M_CFG1_I1A1_IFCLASS      0                 /* Interface class */
#define M_CFG1_I1A1_IFSUBCLASS   0                 /* Interface subclass */
#define M_CFG1_I1A1_IFPROTOCOL   0                 /* Interface protocol */
#define M_CFG1_I1A1_IFI          0                 /* Interface string index */
#define M_CFG1_I1A1_0_EPADDR     0x82              /* Endpoint address (endpoint 2 IN) */
#define M_CFG1_I1A1_0_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG1_I1A1_0_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG1_I1A1_0_INTER      0                 /* Endpoint interval */
#define M_CFG1_I1A1_1_EPADDR     0x02              /* Endpoint address (endpoint 2 OUT) */
#define M_CFG1_I1A1_1_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG1_I1A1_1_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG1_I1A1_1_INTER      0                 /* Endpoint interval */


/* Define values for descriptors in configuration 2 */
#define M_CFG2_NUMIFS            2                 /* Number of interfaces */
#define M_CFG2_CFGI              0                 /* Configuration description string index
*/
#define M_CFG2_ATTRIBUTES        0x00              /* Configuration attributes */
#define M_CFG2_MAXPOWER          0                 /* Maximum power (units of 2 mA) */

/* Interface 0, alternate 0 */
#define M_CFG2_I0A0_NUMEPS       2                 /* Number of endpoints */
#define M_CFG2_I0A0_IFCLASS      0                 /* Interface class */
#define M_CFG2_I0A0_IFSUBCLASS   0                 /* Interface subclass */
#define M_CFG2_I0A0_IFPROTOCOL   0                 /* Interface protocol */
#define M_CFG2_I0A0_IFI          0                 /* Interface string index */
#define M_CFG2_I0A0_0_EPADDR     0x81              /* Endpoint address (endpoint 1 IN) */
#define M_CFG2_I0A0_0_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG2_I0A0_0_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG2_I0A0_0_INTER      0                 /* Endpoint interval */
#define M_CFG2_I0A0_1_EPADDR     0x01              /* Endpoint address (endpoint 1 OUT) */
#define M_CFG2_I0A0_1_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG2_I0A0_1_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG2_I0A0_1_INTER      0                 /* Endpoint interval */

/* Interface 1, alternate 0 */
#define M_CFG2_I1A0_NUMEPS       2                 /* Number of endpoints */
#define M_CFG2_I1A0_IFCLASS      0                 /* Interface class */
#define M_CFG2_I1A0_IFSUBCLASS   0                 /* Interface subclass */
#define M_CFG2_I1A0_IFPROTOCOL   0                 /* Interface protocol */
#define M_CFG2_I1A0_IFI          0                 /* Interface string index */
#define M_CFG2_I1A0_0_EPADDR     0x82              /* Endpoint address (endpoint 2 IN) */
#define M_CFG2_I1A0_0_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG2_I1A0_0_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG2_I1A0_0_INTER      0                 /* Endpoint interval */
#define M_CFG2_I1A0_1_EPADDR     0x02              /* Endpoint address (endpoint 2 OUT) */
#define M_CFG2_I1A0_1_ATTRIB     0x02              /* Endpoint attributes (bulk) */
#define M_CFG2_I1A0_1_MAXP       0x0040            /* Endpoint max packet size (64) */
#define M_CFG2_I1A0_1_INTER      0                 /* Endpoint interval */
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
/*
  Define configuration structures
  The configuration structures need to be modified to match the required configuration
  Each configuration should start with a configuration descriptor
  followed by 1 or more interface descriptors (each of which is followed by
  endpoint descriptors for all the endpoints in that interface).
  The interface descriptors should start with interface 0, alternate setting 0.
  Followed by any alternate settings for interface 0.
  Then interface 1 (if more than 1 interface in the configuration), followed
  by any alternate settings for interface 1.
  This arrangement should be repeated for all interfaces in the configuration.
*/

/* Configuration 1 */
typedef struct M_CFG_1 {
  STD_CFG_DSCR    stdcfg;        /* Required Standard Configuration Descriptor */
  STD_IF_DSCR     stdif00;       /* Interface 0 Descriptor */
  STD_EP_DSCR     stdep00_0;     /* 1st Endpoint Descriptor for Interface 0 */
  STD_EP_DSCR     stdep00_1;     /* 2nd Endpoint Descriptor for Interface 0 */
  STD_IF_DSCR     stdif10;       /* Interface 1 Alternate 0 Descriptor */
  STD_EP_DSCR     stdep10_0;     /* 1st Endpoint Descriptor for Interface 1 */
  STD_EP_DSCR     stdep10_1;     /* 2nd Endpoint Descriptor for Interface 1 */
  STD_IF_DSCR     stdif11;       /* Interface 1 Alternate 1 Descriptor */
  STD_EP_DSCR     stdep11_0;     /* 1st Endpoint Descriptor for Interface 1 */
  STD_EP_DSCR     stdep11_1;     /* 2nd Endpoint Descriptor for Interface 1 */
  } M_CFG_1;
typedef M_CFG_1 * PM_CFG_1;

/* Configuration 2 */
typedef struct M_CFG_2 {
  STD_CFG_DSCR    stdcfg;        /* Required Standard Configuration Descriptor */
  STD_IF_DSCR     stdif00;       /* Interface 0 Descriptor */
  STD_EP_DSCR     stdep00_0;     /* 1st Endpoint Descriptor for Interface 0 */
  STD_EP_DSCR     stdep00_1;     /* 2nd Endpoint Descriptor for Interface 0 */
  STD_IF_DSCR     stdif10;       /* Interface 1 Descriptor */
  STD_EP_DSCR     stdep10_0;     /* 1st Endpoint Descriptor for Interface 1 */
  STD_EP_DSCR     stdep10_1;     /* 2nd Endpoint Descriptor for Interface 1 */
  } M_CFG_2;
typedef M_CFG_2 * PM_CFG_2;

/* Top level structure containing pointers to each configuration */
typedef struct M_CFG {
  PM_CFG_1        pcfg1;         /* Pointer to M_CFG_1 */
  PM_CFG_2        pcfg2;         /* Pointer to M_CFG_2 */
  } M_CFG;
typedef M_CFG * PM_CFG;

#endif /* _CONFIG_H_ */
```


© 2000-2002 Mentor Graphics Corporation.

DESCRIPT.H

```
/****************************************
  descript.h
  Endpoint 0 descriptor definitions for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000

  Note the defined structures assume WORDS are stored little-endian.
****************************************/

#ifndef _DESCRIPT_H_
#define _DESCRIPT_H_

#pragma pack(1)

/* Command structure */
typedef struct COMMAND {
  BYTE    bmRequestType;
  BYTE    bRequest;
  WORD    wValue;
  WORD    wIndex;
  WORD    wLength;
  } COMMAND;
typedef COMMAND * PCOMMAND;

/* Command bit fields */
#define M_CMD_DATADIR            0x80

/* Request Type Field */
#define M_CMD_TYPEMASK           0x60
#define M_CMD_STDREQ             0x00
#define M_CMD_CLASSREQ           0x20
#define M_CMD_VENDREQ            0x40
#define M_CMD_STDDEVIN           0x80
#define M_CMD_STDDEVOUT          0x00
#define M_CMD_STDIFIN            0x81
#define M_CMD_STDIFOUT           0x01
#define M_CMD_STDEPIN            0x82
#define M_CMD_STDEPOUT           0x02

/* Standard Request Codes */
#define GET_STATUS               0x00
#define CLEAR_FEATURE            0x01
#define SET_FEATURE              0x03
#define SET_ADDRESS              0x05
#define GET_DESCRIPTOR           0x06
#define SET_DESCRIPTOR           0x07
#define GET_CONFIGURATION        0x08
#define SET_CONFIGURATION        0x09
#define GET_INTERFACE            0x0A
#define SET_INTERFACE            0x0B
#define SYNCH_FRAME              0x0C

/* Standard Descriptor Types */
#define M_DST_DEVICE             0x01
#define M_DST_CONFIG             0x02
#define M_DST_STRING             0x03
#define M_DST_INTERFACE          0x04
#define M_DST_ENDPOINT           0x05
#define M_DST_DEVQUAL            0x06
#define M_DST_OTHERSPEED         0x07
#define M_DST_POWER              0x08
#define M_CMD_DESCMASK           0xFF00
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
#define M_CMD_DEVICE            (M_DST_DEVICE<<8)
#define M_CMD_CONFIG            (M_DST_CONFIG<<8)
#define M_CMD_STRING            (M_DST_STRING<<8)
#define M_CMD_DEVQUAL           (M_DST_DEVQUAL<<8)
#define M_CMD_OTHERSPEED        (M_DST_OTHERSPEED<<8)

/* Endpoint transfer types */
#define M_EP_TFMASK             0x03
#define M_EP_CONTROL            0x00
#define M_EP_ISO                0x01
#define M_EP_BULK               0x10
#define M_EP_INTR               0x11

/* Standard Device Feature Selectors */
#define M_FTR_DEVREMWAKE        0x0001

/* Standard Device Descriptor */
typedef struct STD_DEV_DSCR {
  BYTE    bLength;
  BYTE    bDescriptorType;
  WORD    bcdUSB;
  BYTE    bDeviceClass;
  BYTE    bDeviceSubClass;
  BYTE    bDeviceProtocol;
  BYTE    bMaxPacketSize0;
  WORD    idVendor;
  WORD    idProduct;
  WORD    bcdDevice;
  BYTE    iManufacturer;
  BYTE    iProduct;
  BYTE    iSerialNumber;
  BYTE    bNumConfigurations;
  } STD_DEV_DSCR;
typedef STD_DEV_DSCR * PSTD_DEV_DSCR;

/* Standard Configuration Descriptor */
typedef struct STD_CFG_DSCR {
  BYTE    bLength;
  BYTE    bDescriptorType;
  WORD    wTotalLength;
  BYTE    bNumInterfaces;
  BYTE    bConfigurationValue;
  BYTE    iConfiguration;
  BYTE    bmAttributes;
  BYTE    bMaxPower;
  } STD_CFG_DSCR;
typedef STD_CFG_DSCR * PSTD_CFG_DSCR;

/* Standard Interface Descriptor */
typedef struct STD_IF_DSCR {
  BYTE    bLength;
  BYTE    bDescriptorType;
  BYTE    bInterfaceNumber;
  BYTE    bAlternateSetting;
  BYTE    bNumEndpoints;
  BYTE    bInterfaceClass;
  BYTE    bInterfaceSubClass;
  BYTE    bInterfaceProtocol;
  BYTE    iInterface;
  } STD_IF_DSCR;
typedef STD_IF_DSCR * PSTD_IF_DSCR;
```


© 2000-2002 Mentor Graphics Corporation.

```
/* Standard Endpoint Descriptor */
typedef struct STD_EP_DSCR {
  BYTE    bLength;
  BYTE    bDescriptorType;
  BYTE    bEndpointAddress;
  BYTE    bmAttributes;
  WORD    wMaxPacketSize;
  BYTE    bInterval;
  } STD_EP_DSCR;
typedef STD_EP_DSCR * PSTD_EP_DSCR;


#endif /* _DESCRIPT_H_ */
```

ENDPOINT0.H

```
/****************************************

  endpoint0.h
  Endpoint 0 header file for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000

****************************************/

#ifndef _ENDPOINT0_H_
#define _ENDPOINT0_H_


/* Endpoint 0 states */
#define M_EP0_IDLE  0
#define M_EP0_RX    1
#define M_EP0_TX    2

/* Endpoint 0 status structure */
typedef struct M_EP0_STATUS {
  int     nState;               /* IDLE/RX/TX */
  int     nBytesLeft;           /* Number of bytes left to send in TX mode */
  void   *pData;                /* Pointer to data to transmit/receive */
  int     nBytesRecv;           /* Number of bytes received in RX mode */
  BYTE    byFAddr;              /* New function address */
  } M_EP0_STATUS;
typedef M_EP0_STATUS * PM_EP0_STATUS;

#endif /* _ENDPOINT0_H_ */
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

ENDPOINT.H

```
/****************************************
  endpoint.h
  Endpoint header file for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/

#ifndef _ENDPOINT_H_
#define _ENDPOINT_H_


#define M_EP_NODATA -1

/* Define maximum packet size for bulk endpoints */
#define M_EP_MAXP   64


/* Bulk In Endpoint status structure */
typedef struct M_EPBIN_STATUS {
  void   *pData;                /* Pointer to data to transmit */
  int     nBytesLeft;           /* Number of bytes left to send */
  BYTE    byEP;                 /* Endpoint number */
  } M_EPBIN_STATUS;
typedef M_EPBIN_STATUS * PM_EPBIN_STATUS;

/* Bulk Out Endpoint status structure */
typedef struct M_EPBOUT_STATUS {
  void   *pData;                /* Pointer to data buffer */
  int     nBuffLen;             /* Length of data buffer */
  int     nBytesRecv;           /* Number of bytes received */
  BYTE    byEP;                 /* Endpoint number */
  } M_EPBOUT_STATUS;
typedef M_EPBOUT_STATUS * PM_EPBOUT_STATUS;

/* Function prototypes */
void EndpointBulkIn (PM_EPBIN_STATUS, int);
void EndpointBulkOut (PM_EPBOUT_STATUS, int);

#endif /* _ENDPOINT_H_ */
```

MUSBFSFC.H

```
/****************************************
  musbfsfc.h
  Header file for MUSBFSFC firmware

  Copyright Mentor Graphics Corporation and Licensors 2000
  V1.0  4 September 2000
****************************************/

#ifndef _MUSBFSFC_H_
#define _MUSBFSFC_H_

/* Define MUSBFSFC register addresses */
#define BASE_ADDR 0

#define M_REG_FADDR             BASE_ADDR
#define M_REG_POWER             BASE_ADDR+1
#define M_REG_INTRIN1           BASE_ADDR+2
```


© 2000-2002 Mentor Graphics Corporation.

```
#define M_REG_INTRIN2            BASE_ADDR+3
#define M_REG_INTROUT1           BASE_ADDR+4
#define M_REG_INTROUT2           BASE_ADDR+5
#define M_REG_INTRUSB            BASE_ADDR+6
#define M_REG_INTRIN1E           BASE_ADDR+7
#define M_REG_INTRIN2E           BASE_ADDR+8
#define M_REG_INTROUT1E          BASE_ADDR+9
#define M_REG_INTROUT2E          BASE_ADDR+10
#define M_REG_INTRUSBE           BASE_ADDR+11
#define M_REG_FRAME1             BASE_ADDR+12
#define M_REG_FRAME2             BASE_ADDR+13
#define M_REG_INDEX              BASE_ADDR+14

#define M_REG_CSR0               BASE_ADDR+17
#define M_REG_INMAXP             BASE_ADDR+16
#define M_REG_INCSR1             BASE_ADDR+17
#define M_REG_INCSR2             BASE_ADDR+18
#define M_REG_OUTMAXP            BASE_ADDR+19
#define M_REG_OUTCSR1            BASE_ADDR+20
#define M_REG_OUTCSR2            BASE_ADDR+21
#define M_REG_OUTCOUNT1          BASE_ADDR+22
#define M_REG_OUTCOUNT2          BASE_ADDR+23

#define M_FIFO_EP0               BASE_ADDR+32


/* Interrupt register bit masks */
#define M_INTR_SUSPEND           0x01
#define M_INTR_RESUME            0x02
#define M_INTR_RESET             0x04
#define M_INTR_EP0               0x0001

/* CSR0 bit masks */
#define M_CSR0_OUTPKTRDY         0x01
#define M_CSR0_INPKTRDY          0x02
#define M_CSR0_SENTSTALL         0x04
#define M_CSR0_DATAEND           0x08
#define M_CSR0_SETUPEND          0x10
#define M_CSR0_SENDSTALL         0x20
#define M_CSR0_SVDOUTPKTRDY      0x40
#define M_CSR0_SVDSETUPEND       0x80

/* Endpoint CSR register bits */
#define M_INCSR2_ISO             0x40
#define M_INCSR2_MODE            0x20
#define M_INCSR_CDT              0x40
#define M_INCSR_FF               0x08
#define M_INCSR_IPR              0x01
#define M_OUTCSR2_ISO            0x40
#define M_OUTCSR_CDT             0x80
#define M_OUTCSR_FF              0x10
#define M_OUTCSR_OPR             0x01


/* Define integer types for 8/16 bit registers */
#ifndef BYTE
typedef unsigned char BYTE;
#endif
#ifndef WORD
typedef unsigned short WORD;
#endif


/* Register read/write macros */
#define MREAD_BYTE(addr) *((BYTE *)(addr))
#define MWRITE_BYTE(addr,data) *((BYTE *)(addr)) = data
```

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

```
/* Define endpoint call status values */
#define M_EP_NORMAL              0
#define M_EP_RESET               1

/* Define device states */
#define DEVSTATE_DEFAULT         0
#define DEVSTATE_ADDRESS         1
#define DEVSTATE_CONFIG          2

/* Define standard constants */
#define TRUE  1
#define FALSE 0
#ifndef NULL
#define NULL 0
#endif

/* Function prototypes */
void FIFORead (int, int, void*);
void FIFOWrite (int, int, void*);
void USBReset ();
void InitialiseDesc ();
void Endpoint0 (int);

#endif /* _MUSBFSFC_H_ */
```


© 2000-2002 Mentor Graphics Corporation.

15. REVISION HISTORY

15.1. ISSUE 1

15th March 2001. First issue of document.

15.2. ISSUE 2

23rd May 2001. Changes made following addition of 32- or 64-byte FIFO options for Endpoint 0 to the design.

15.3. ISSUE 3

14th March 2002. Modified following the inclusion of optional bridges such as an MUSBFSFC – AMBA AHB bridge with the MUSBFSFC design.

CONFIDENTIAL

© 2000-2002 Mentor Graphics Corporation.

© 2000-2002 Mentor Graphics Corporation, All Rights Reserved.
™Mentor Graphics and Inventra are trademarks of Mentor Graphics Corporation.
All other trademarks are the property of their respective owners.

http://www.mentor.com/inventra

Corporate Headquarters
Mentor Graphics Corporation
8005 S.W. Boeckman Road
Wilsonville, OR 97070 USA
Phone: 503-685-7000
North American Support Center
Phone: 800-547-4303
Fax: 800-684-1795

Silicon Valley Headquarters
Mentor Graphics Corporation
1001 Ridder Park Drive
San Jose, California 95131 USA
Phone: 408-436-1500
Fax: 408-436-1501

Europe Headquarters
Mentor Graphics Corporation
Immeuble le Pasteur
13/15, rue Jeanne Braconnier
92360 Meudon La Foret
France
Phone: 33-1-40-94-74-74
Fax: 33-1-46-01-91-73

Pacific Rim Headquarters
Mentor Graphics (Taiwan)
Room 1603, 16F,
International Trade Building
No.333, Section 1, Keelung Road
Taipei, Taiwan, ROC
Phone: 886-2-87252000
Fax: 886-2-27576027

Japan Headquarters
Mentor Graphics Japan Co., Ltd.
Gotenyama Hills
7-35, Kita-Shinagawa 4-chome
Shinagawa-Ku, Tokyo 140
Japan
Phone: 81-3-5488-3033
Fax: 81-3-5488-3021

03/02 PG-40104.003-FC